



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

FRANCESCO STIRO – 1000027019

MARIO TOSCANO – 1000044143

AUTOMATIC DETECTION AND COUNTING OF
PEOPLE IN IMAGES AND VIDEOS: A DEEP
LEARNING APPROACH

MACHINE LEARNING - PROGETTO FINALE

Docenti:

Prof. Giovanni Maria Farinella, Prof. Rosario Leonardi

ANNO ACCADEMICO 2024/2025

Contents

1	Introduzione	2
2	Object Detection	4
2.1	Applicazioni reali	5
2.1.1	Medical Image Analysis	5
2.1.2	Quality Control	5
2.1.3	Crowd Counting and Traffic Monitoring	5
2.2	YOLO - "You Only Look Once"	6
2.2.1	Evoluzioni di YOLO	7
2.2.2	YOLO12	8
2.3	Struttura architetturale di YOLO12	9
2.3.1	Backbone	11
2.3.2	Neck	12
2.3.3	Head	12
2.3.4	Funzione di perdita in YOLO12	13
3	Dataset	15
3.1	Dataset di Training	15
3.1.1	MOT17	15
3.1.2	MOT20	16
3.1.3	CrowdHuman	17
3.1.4	CUHK-Pedestrian	18
3.1.5	Crowd TypSA (Escluso)	18
3.2	Preprocessing e Conversione	19
3.2.1	Formato delle Annotazioni in YOLO12	19
3.3	Distribuzione spaziale delle annotazioni	19
3.3.1	Sampling dei Frame	21
3.3.2	Struttura delle Cartelle e Validazione	21
3.4	Validation set	22
3.5	Test Set	23
3.6	Statistiche del Dataset di Test	24

4	Metodologia	27
4.1	Preprocessing dei dati	28
4.2	Training e Setup Sperimentale	30
4.2.1	Configurazione dell'Addestramento	30
4.2.2	Dettagli dell'Ottimizzazione	31
4.2.3	Ambiente di Esecuzione	31
4.2.4	Dataset e Considerazioni Pratiche	32
4.2.5	Strategie di generalizzazione	32
4.2.6	Utilizzo di SAHI (Slicing Aided Hyper Inference)	33
5	Metriche di valutazione per l'Object Detection	36
5.1	Intersection over Union (IoU)	36
5.2	Precisione e Richiamo (Precision & Recall)	37
5.3	F1-score	38
5.4	Average Precision e Mean Average Precision	38
5.4.1	Considerazioni sul ruolo della soglia IoU in Object De- tection	40
5.5	Loss functions	41
5.6	Matrice di Confusione	42
5.6.1	Considerazioni sulla matrice di confusione nel caso mon- oclasse	43
6	Risultati Sperimentali	45
6.1	YOLO12 Training Data	45
6.1.1	YOLO12-n	46
6.1.2	YOLO12-s	47
6.1.3	YOLO12-m	49
6.2	Confronto tra Modelli	50
6.3	Valutazione sul Test Set	50
6.3.1	YOLO12-n e YOLO12-n (fine-tuned)	51
6.3.2	YOLO12-s e YOLO12-s (fine-tuned)	53
6.3.3	YOLO12-m e YOLO12-m (fine-tuned)	54
6.3.4	Tabella riepilogativa finale	55
7	Demo	58
7.1	Interfaccia Grafica	58
7.1.1	Visualizzazione dei Risultati	59
7.1.2	Controlli Principali	60
7.1.3	Controlli successivi ad una predizione	61
7.1.4	Elaborazione Video	62

8	Conclusioni	64
8.1	Risultati Ottenuti	64
8.2	Lezioni Apprese	65
8.3	Sviluppi Futuri	65
9	Codice	67
A	Script di Conversione per il Dataset CrowdHuman	70
B	Script di Conversione per il dataset CUHk	72
C	Script di Conversione per il dataset MOT	76
D	Generazione delle heatmap per training e validation	79
E	Procedura di Training	81
	Bibliography	83

Chapter 1

Introduzione

L'object detection ha assunto un ruolo centrale nel campo della computer vision e del machine learning. Si tratta di una tecnica che consiste non solo di rilevare la presenza di oggetti all'interno di un'immagine, ma anche di localizzarli precisamente mediante l'utilizzo di un bounding box. Tra i diversi algoritmi sviluppati per la object detection, la famiglia di algoritmi YOLO si è distinta principalmente per la sua capacità di distinguere oggetti in tempo reale. YOLO12 è il modello di riferimento allo stato dell'arte per la object detection. Questo modello utilizza un'implementazione di YOLO basata sull'attenzione, che secondo il paper di riferimento “eguaglia la velocità dei precedenti modelli basati su reti neurali convoluzionali (CNN), pur sfruttando i vantaggi prestazionali dei meccanismi di attenzione”.

Esistono diverse size del modello YOLO12:

- YOLO12n: Ideale per dispositivi edge o applicazioni in tempo reale con risorse limitate.
- YOLO12s: Un buon compromesso tra velocità e accuratezza per applicazioni mobile o web.
- YOLO12m: Adatto per sistemi con GPU di fascia media, offrendo un equilibrio tra prestazioni e requisiti computazionali.
- YOLO12l: Modello grande, progettato per offrire alta accuratezza mantenendo un buon compromesso con la velocità di inferenza.

- YOLO12x: Modello extra-large, progettato per massimizzare la precisione, ma è molto più pesante e lento ad inferire.

Modello	dimensione (pixel)	mAPval 50-95	Velocità CPU ONNX (ms)	Velocità T4 TensorRT (ms)	params (M)	FLOP (B)	Confronto (mAP/velocità)
YOLO12n	640	40.6	-	1.64	2.6	6.5	+2,1%/-9% (rispetto a YOLOv10n)
YOLO12s	640	48.0	-	2.61	9.3	21.4	+0,1%/+42% (rispetto a RT-DETRv2)
YOLO12m	640	52.5	-	4.86	20.2	67.5	+1,0%/-3% (vs. YOLO11m)
YOLO12l	640	53.7	-	6.77	26.4	88.9	+0,4%/-8% (vs. YOLO11l)
YOLO12x	640	55.2	-	11.79	59.1	199.0	+0,6%/-4% (vs. YOLO11x)

Figure 1.1: Prestazioni di rilevamento dei diversi modelli

Il progetto ha come obiettivo la realizzazione di un sistema automatico per il conteggio delle persone in una scena tramite tecniche di object detection. In particolare, si è scelto di utilizzare YOLO12, un modello di ultima generazione per la rilevazione in tempo reale di oggetti, noto per l'elevata accuratezza e la velocità di inferenza.

L'idea alla base del progetto è rilevare tutte le persone presenti in un'immagine o in un video, e successivamente contarle. La pipeline del sistema può quindi essere suddivisa in due fasi principali:

1. Rilevamento: Individuare tutte le istanze della classe "persona" nella scena. Questa classe sarà l'aggregazione di diverse sottoclassi che rappresentano l
2. Conteggio: contare il numero totale di rilevamenti validi relativi alla classe "persona".

Si vorranno poi confrontare le prestazioni dei vari modelli di YOLO12 elencati precedentemente. La valutazione delle performance dei modelli sarà effettuata sulla base di apposite metriche di valutazione di cui si parlerà approfonditamente nelle apposite sezioni.

Chapter 2

Object Detection

L'object detection (o rilevamento di oggetti) è uno dei task fondamentali nel campo della visione artificiale. Esso consiste nel rilevare istanze di oggetti semantici di una data classe (ad esempio esseri umani - come nel caso del presente progetto - veicoli, edifici, etc). Un algoritmo di object detection può essere visto come una combinazione di due sottoproblemi:

1. **Regressione** per predire le coordinate delle bounding box (rettangoli di delimitazione) che racchiudono gli oggetti nelle immagini.
2. **Classificazione** per determinare la categoria a cui ciascun oggetto appartiene.

In questo senso, un modello di object detection esegue simultaneamente una regressione spaziale (per la localizzazione) e una classificazione (per l'identificazione semantica). Le principali sfide dell'object detection includono:

- **Variabilità di scala:** gli oggetti possono apparire molto grandi o molto piccoli, a seconda della distanza dalla telecamera.
- **Occlusioni parziali o totali di oggetti:** ad esempio in ambienti affollati, le persone possono sovrapporsi.
- **Condizioni di illuminazione non uniformi:** luci forti, ombre o riflessi possono rendere difficile il rilevamento.

- **Complessità dello sfondo:** la presenza di elementi non rilevanti o visivamente simili agli oggetti target può aumentare la probabilità di falsi positivi o negativi.

Nelle sezioni successive vedremo come abbiamo cercato di gestire queste problematiche per fare in modo che il modello riesca a generalizzare meglio.

2.1 Applicazioni reali

L'object detection ha trovato impiego in numerosi ambiti applicativi, grazie alla sua capacità di analizzare visivamente la scena in modo automatico ed efficiente. Tra le applicazioni più rilevanti si possono citare le seguenti:

2.1.1 Medical Image Analysis

Secondo l'IBM, le immagini costituiscono il 90% di tutti i dati medici. Le tecnologie di apprendimento automatico capaci di analizzare immagini possono identificare anomalie e oggetti osservabili e rappresentano un potenziale enorme come strumento di supporto per i medici. Questo permetterebbe agli specialisti sanitari di individuare schemi che sarebbero estremamente difficili da rilevare anche per i professionisti più esperti.

2.1.2 Quality Control

Le aziende possono utilizzare le tecnologie di object detection per trovare difetti nei materiali o nei loro prodotti, garantendo una qualità migliore e automatizzando il processo.

2.1.3 Crowd Counting and Traffic Monitoring

E' possibile contare il numero di persone in aree pubbliche, oppure i veicoli su strade o percorsi specifici, o ancora rilevare il tipo di azione svolta dai soggetti; quest'ultima può essere integrata nei sistemi di videosorveglianza per aumentare la sicurezza. E' possibile inoltre usare le informazioni ottenute per un processing ulteriore al fine di prendere decisioni in maniera automatizzata. Ci sarebbero moltissime altre applicazioni che per brevità

non citiamo, ma è interessante notare che un task come la object detection può essere applicato a contesti e situazioni molto diverse tra loro.

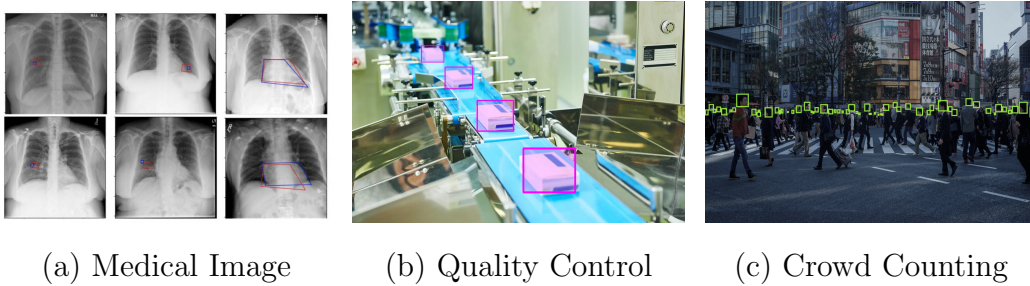


Figure 2.1: Applicazioni Reali

Nel caso specifico di questo progetto, l'obiettivo che ci siamo posti è stato di sviluppare un sistema automatico in grado di rilevare e contare il numero di persone presenti in ogni frame di un video, con particolare riferimento a scenari reali caratterizzati da densità moderata di pedoni. Il sistema è progettato per operare efficacemente in ambienti mediamente affollati, in cui le persone possono essere parzialmente occluse ma non completamente sovrapposte in massa, come accade nei contesti di sovraffollamento estremo.

2.2 YOLO - "You Only Look Once"

Il nome YOLO (You Only Look Once) riflette il principio fondamentale dell'architettura: l'intera immagine viene analizzata una sola volta dalla rete neurale per predire simultaneamente le bounding box e le classi degli oggetti presenti. Il problema viene quindi formulato come un problema di regressione e classificazione unico e globale. L'output, infatti, fornisce simultaneamente le bounding box, le classi e i punteggi di confidenza: tutto in un'unica passata. A differenza di approcci precedenti, come R-CNN o Fast R-CNN, che suddividevano il processo in più fasi (proposta di regioni, classificazione, regressione), YOLO unifica il tutto in un'unica elaborazione end-to-end. Questa caratteristica consente prestazioni estremamente rapide, rendendo YOLO particolarmente adatto per applicazioni in **tempo reale**.

2.2.1 Evoluzioni di YOLO

Sin dalla sua introduzione, la famiglia YOLO è stata un punto di riferimento per la object detection in tempo reale, grazie alla capacità di predire simultaneamente bounding box e classi in un'unica passata sulla rete. A partire da YOLOv1, ogni versione successiva ha apportato miglioramenti in termini di velocità e precisione.

Le prime evoluzioni (YOLOv2, v3) hanno introdotto l'estrazione multi-scala delle feature e strategie di training più avanzate. YOLOv4–v6 hanno ottimizzato ulteriormente l'equilibrio tra efficienza computazionale e precisione, usando tecniche come mosaic augmentation e CSPNet. Le versioni YOLOv7–v9 hanno migliorato la versatilità su diversi tipi di hardware, da dispositivi edge a GPU ad alte prestazioni. Con YOLOv10 e v11 sono stati integrati meccanismi di attenzione e componenti ispirati ai transformer, migliorando il riconoscimento di pattern complessi. Tuttavia, la rilevazione precisa di oggetti piccoli, sovrapposti o parzialmente occlusi, in tempo reale, restava una sfida aperta. Tuttavia, con YOLO12 si compie un nuovo salto qualitativo.

Release	Year	Tasks	Contributions	Framework
YOLO [1]	2015	Object Detection, Basic Classification	Single-stage object detector	Darknet
YOLOv2 [6]	2016	Object Detection, Enhanced Classification	Multi-scale training, dimension clustering	Darknet
YOLOv3 [13]	2018	Object Detection, Multi-scale	SPP block, Darknet-53 backbone	Darknet
YOLOv4 [12]	2020	Object Detection, Basic Tracking	Mish activation, CSPDarknet-53 backbone	Darknet
YOLOv5 [14]	2020	Object Detection, Instance Segmentation	Anchor-free detection, SWISH activation, PANet	PyTorch
YOLOv6 [15]	2022	Object Detection, Instance Segmentation	Self-attention, anchor-free OD	PyTorch
YOLOv7 [16]	2022	Object Detection, Tracking, Segmentation	Transformers, E-ELAN reparameterization	PyTorch
YOLOv8 [17]	2023	Object Detection, Instance and Panoptic Segmentation	GANs, anchor-free detection	PyTorch
YOLOv9 [18]	2024	Object Detection, Instance Segmentation	PGI and GELAN	PyTorch
YOLOv10 [19]	2024	Object Detection	Consistent dual assignments for NMS-free training	PyTorch
YOLOv11 [10]	2024	Object Detection, Instance Segmentation	Expanded capabilities, improved efficiency	PyTorch
YOLOv12 [11]	2025	Object Detection, Instance Segmentation	Advanced attention-centric design (FlashAttention, R-ELAN), 7 × 7 separable convolutions, improved computational efficiency	PyTorch

Figure 2.2: Comparazione dei diversi modelli di YOLO

2.2.2 YOLO12

YOLO12 rappresenta un'evoluzione significativa nell'ambito della *object detection* in tempo reale, proponendosi come una svolta di paradigma grazie all'integrazione di **meccanismi attention-centrici**, **design architetturali ottimizzati** e **pipeline di training raffinate**. A partire dalle solide basi delle versioni precedenti, YOLO12 introduce una serie di innovazioni orientate a **massimizzare la precisione e l'efficienza computazionale**.

Al centro di questa versione si trova una strategia di estrazione delle feature completamente riprogettata, che fa uso del modulo **R-ELAN (Residual Efficient Layer Aggregation Network)**, illustrato nella Figura 2.3. Questo modulo, attraverso una struttura gerarchica di blocchi convoluzionali aggregati e connessioni residue, migliora la trasmissione del gradiente e favorisce una fusione più efficace delle informazioni visive. La rete include blocchi ripetuti $A2 \times 2$, convoluzioni 1×1 , operazioni di *concatenation* e un passaggio finale di *scaling residuo*, che garantisce stabilità e potenza espressiva.

In parallelo, YOLO12 introduce un raffinato modulo di **area attention**, il quale suddivide le *feature map* in regioni localizzate e consente al modello di concentrarsi su parti cruciali dell'immagine, soprattutto in presenza di scene complesse, dinamiche o affollate. Questo modulo è accelerato tramite **FlashAttention**, una tecnica che riduce significativamente l'overhead computazionale e il consumo di memoria, permettendo un'elaborazione rapida anche su dispositivi a risorse limitate.

Un'ulteriore innovazione riguarda la sostituzione dei classici *positional encoding* con **convoluzioni separabili 7×7** . Queste operazioni permettono di preservare la consapevolezza spaziale nella rete mantenendo al contempo un numero contenuto di parametri, migliorando così l'efficienza senza sacrificare la precisione.

Grazie a questa combinazione di elementi — R-ELAN, area attention, FlashAttention e convoluzioni separabili — YOLO12 si dimostra particolarmente adatto a scenari **in tempo reale**, dove è necessario bilanciare accuratezza e latenza. La sua architettura è progettata per affrontare con efficacia anche compiti complessi come la *rilevazione di oggetti piccoli, parzialmente*

occlusi o sovrapposti, che in precedenza costituivano una sfida per molti detector.

Inoltre, la versatilità di YOLO12 ne consente l'impiego in diversi contesti applicativi: dai **veicoli autonomi** ai sistemi di **videosorveglianza urbana**, dall'**analisi di immagini mediche** all'**agricoltura di precisione**. In tutti questi casi, il modello è in grado di fornire prestazioni elevate sia in termini di accuratezza che di efficienza operativa, rendendolo uno strumento di riferimento per la *object detection* su larga scala.

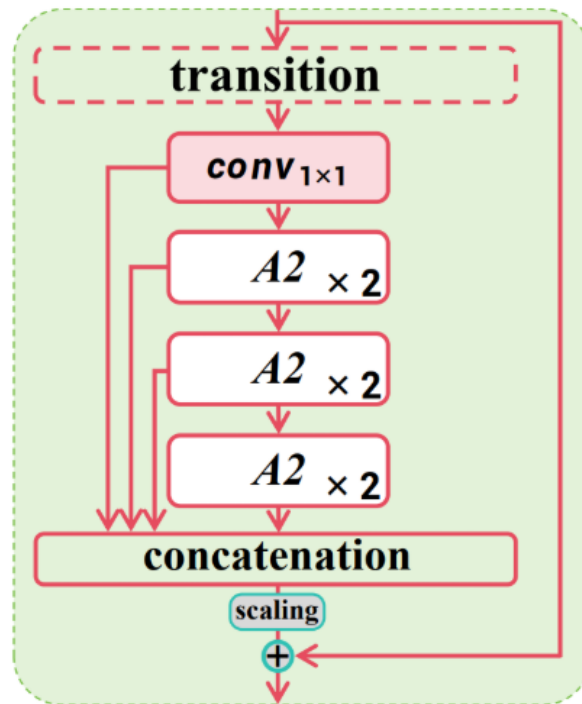


Figure 2.3: Residual Efficient Layer Aggregation Network (R-ELAN), come presentato nel paper ufficiale di YOLO12.

2.3 Struttura architetturale di YOLO12

Il successo della famiglia di modelli YOLO è sempre dipeso da un'architettura unificata in grado di eseguire simultaneamente la *regressione delle bounding box* e la *classificazione degli oggetti*, attraverso un training end-to-end completamente differenziabile. YOLO12 prosegue questa tradizione, estendendola

grazie all'integrazione di innovazioni architetturali progettate specificamente per aumentare la precisione, ridurre la latenza e migliorare l'adattabilità del modello.

L'architettura di YOLO12 può essere suddivisa in tre componenti principali, come mostrato nella Tabella 2.1:

- **Backbone:** estrae le *feature* dalle immagini in input a diverse scale utilizzando layer convoluzionali;
- **Neck:** aggrega e raffina le feature provenienti da diversi livelli della rete, preparandole per la fase di predizione;
- **Head:** genera le predizioni finali, incluse le coordinate delle bounding box e le etichette di classe.

In YOLO12, ciascuna di queste componenti è stata ottimizzata con soluzioni specifiche:

- Il **backbone** sfrutta il modulo **R-ELAN** per migliorare la connettività residua, ed impiega *convoluzioni separabili* 7×7 per preservare il contesto spaziale riducendo al contempo il numero di parametri.
- Il **neck** adotta meccanismi di **area attention**, accelerati tramite **FlashAttention**, per enfatizzare le regioni visivamente più rilevanti all'interno dell'immagine.
- L'**head** include percorsi di predizione ottimizzati per garantire accuratezza nella rilevazione multi-scala, con funzioni di perdita progettate per massimizzare l'efficienza in scenari real-time.

Table 2.1: Componenti strutturali principali di YOLO12

Componente	Funzionalità	Innovazioni in YOLO12
Backbone	Estrae feature da immagini di input a diverse scale tramite layer convoluzionali	R-ELAN per connettività residua profonda; convoluzioni separabili 7×7 per preservare il contesto spaziale con meno parametri
Neck	Aggrega feature da diverse scale e le trasmette alla testa predittiva	Meccanismi di area attention accelerati da FlashAttention per una focalizzazione efficiente sulle regioni critiche
Head	Genera predizioni finali, inclusi bounding box e classi	Percorsi predittivi raffinati per una rilevazione multi-scala accurata; funzioni di perdita ottimizzate per prestazioni real-time

2.3.1 Backbone

Il *backbone* di YOLO12 è responsabile della conversione dei dati grezzi dell'immagine in *feature map* multi-scala, costituendo la base per i successivi task di rilevamento. Al centro di questa struttura si trova il modulo **R-ELAN (Residual Efficient Layer Aggregation Network)**, che fonde blocchi convoluzionali profondi con connessioni residue attentamente progettate. Questo approccio riduce i colli di bottiglia nei gradienti e migliora la qualità delle feature, permettendo al modello di catturare dettagli complessi di oggetti di varie forme e dimensioni.

Blocchi convoluzionali avanzati

YOLO12 introduce una nuova classe di blocchi convoluzionali leggeri, progettati per una maggiore parallelizzazione. Tali blocchi utilizzano una serie di kernel di piccole dimensioni anziché kernel grandi e complessi, ottenendo una maggiore efficienza computazionale senza compromettere la qualità della rappresentazione. Il blocco è rappresentabile genericamente dalla formula:

$$F_{\text{out}} = \sum_{i=1}^n W_i * F_{\text{in}} + b_i \quad (2.1)$$

dove F_{in} è la feature map in input, W_i sono i filtri convoluzionali, b_i è il bias, e F_{out} rappresenta la feature map in output.

Architettura avanzata del backbone

Oltre ai nuovi blocchi convoluzionali, YOLO12 impiega anche le **convoluzioni separabili** 7×7 , che sostituiscono efficacemente le operazioni convoluzionali ampie o i classici encoding posizionali. Questo consente di mantenere la consapevolezza spaziale con meno parametri. Inoltre, l'utilizzo di *multi-scale feature pyramids* assicura che oggetti di varie dimensioni, inclusi quelli piccoli o parzialmente occlusi, siano rappresentati in modo distinto all'interno della rete.

2.3.2 Neck

Il *neck* agisce come ponte tra il backbone e l'head, aggregando e raffinando feature multi-scala. Una delle innovazioni chiave è l'utilizzo del meccanismo di **area attention**, accelerato da **FlashAttention**, che consente al modello di concentrarsi su regioni critiche in ambienti complessi. Questo meccanismo è descritto matematicamente come un'operazione di attenzione segmentata:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.2)$$

dove Q , K , V sono rispettivamente le matrici di query, key e value, e d_k è la dimensionalità delle key. Segmentando le feature map e applicando l'attenzione in modo efficiente, YOLO12 riduce il carico computazionale e i trasferimenti di memoria, garantendo prestazioni real-time anche a risoluzioni elevate.

2.3.3 Head

L'*head* di YOLO12 trasforma le feature map raffinate in predizioni finali, generando le coordinate delle bounding box e le etichette di classe. Le principali innovazioni includono:

- percorsi predittivi ottimizzati per la rilevazione multi-scala,

- funzioni di perdita specializzate che bilanciano localizzazione e classificazione.

2.3.4 Funzione di perdita in YOLO12

Una delle componenti fondamentali per l'addestramento efficace di YOLO12 è la **funzione di perdita**, che guida l'ottimizzazione della rete neurale. Tradizionalmente, nei modelli YOLO più semplici (ad esempio YOLOv1), la funzione di perdita viene espressa come una combinazione tra la perdita di localizzazione, la perdita di confidenza e la perdita di classificazione. Una versione semplificata, utile a scopo illustrativo, è la seguente:

$$\mathcal{L} = \lambda_{\text{coord}} \sum ((\hat{x} - x)^2 + (\hat{y} - y)^2) + \lambda_{\text{obj}} \sum (\hat{C} - C)^2 \quad (2.3)$$

dove:

- $\hat{x}, \hat{y}$ sono le coordinate predette del centro della bounding box;
- $\hat{C}$ è il punteggio di confidenza;
- x, y, C sono i valori reali (ground truth);
- $\lambda_{\text{coord}}, \lambda_{\text{obj}}$ sono pesi di bilanciamento tra le varie componenti.

Tuttavia, questa formulazione è ormai considerata **inadeguata** per le versioni più avanzate del modello, come YOLO12, che introducono una struttura di perdita più sofisticata e robusta, capace di migliorare sensibilmente la precisione della detection.

Perdita totale in YOLO12

In YOLO12, la funzione di perdita si compone di tre principali componenti:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \cdot \mathcal{L}_{\text{IoU}} + \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}} \quad (2.4)$$

dove:

- $\mathcal{L}_{\text{IoU}}$: perdita di localizzazione, basata su **GIoU** (Generalized Intersection over Union), che valuta l'allineamento tra la bounding box predetta e quella reale:

$$\mathcal{L}_{\text{IoU}} = 1 - \text{GIoU}(\hat{b}, b)$$

- $\mathcal{L}_{\text{cls}}$: perdita di classificazione, solitamente calcolata tramite **cross-entropy** o **focal loss**, che misura la distanza tra le probabilità predette $\hat{p}_c$ e le etichette reali y_c :

$$\mathcal{L}_{\text{cls}} = - \sum_{c=1}^C y_c \log(\hat{p}_c)$$

- $\mathcal{L}_{\text{dff}}$: **Distribution Focal Loss**, un termine avanzato introdotto per predire le coordinate come distribuzioni anziché valori scalari, migliorando la precisione sub-pixel:

$$\mathcal{L}_{\text{dff}} = \sum_{i=1}^4 \text{DFL}(\hat{d}_i, d_i)$$

dove $\hat{d}_i$ è la distribuzione predetta per una coordinata della bounding box, e d_i il valore reale corrispondente.

I coefficienti (iper-parametri) λ_{box} , λ_{cls} , λ_{dff} bilanciano il contributo di ciascun termine. Valori tipici usati nei modelli YOLO12 sono:

$$\lambda_{\text{box}} = 7.5, \quad \lambda_{\text{cls}} = 0.5, \quad \lambda_{\text{dff}} = 1.5$$

Questa funzione di perdita avanzata consente al modello YOLO12 di ottenere una localizzazione più precisa delle bounding box, una classificazione più robusta, e una maggiore stabilità durante l'addestramento. In particolare, la combinazione di **GIoU** e **DFL** consente di affrontare con successo scenari complessi come oggetti piccoli, sovrapposti o occlusi, mantenendo le prestazioni in tempo reale che caratterizzano la famiglia YOLO.

Chapter 3

Dataset

In questa sezione vengono descritti i dataset utilizzati per l'addestramento e la valutazione del modello YOLO12. In particolare, per la fase di **training** sono stati impiegati i dataset **MOT17**, **MOT20**, **CrowdHuman**, **CUHK-Pedestrian** e, inizialmente, anche **Crowd TypSA**, un dataset poi escluso per la scarsa qualità delle annotazioni. È importante sottolineare che il dataset utilizzato per l'addestramento non è stato ottenuto tramite un semplice processo di aggregazione automatica dei dataset pubblici riportati in seguito. Al contrario, le immagini sono state selezionate in modo oculato e mirato, al fine di garantire un livello qualitativo adeguato alle esigenze specifiche del nostro task. In particolare, sono stati privilegiate immagini con annotazioni consistenti e affidabili, evitando immagini eccessivamente degradate, ambigue o ridondanti. Questo processo di curazione ha contribuito in modo significativo alla qualità dell'addestramento e alla capacità del modello di generalizzare in scenari reali di affollamento moderato. Abbiamo comunque mantenuto il training set eterogeneo, mantenendo anche immagini ad alta densità affinché il modello potesse performare adeguatamente anche in scenari differenti.

3.1 Dataset di Training

3.1.1 MOT17

Il dataset **MOT17** (*Multiple Object Tracking Challenge 2017*) [1] include 7 video etichettati e 7 non etichettati, per un totale di oltre 11.000 frame

annotati manualmente. Le scene coprono ambienti urbani dinamici, con telecamere fisse o in movimento. Ogni persona presente nel frame è etichettata tramite una *bounding box* e un ID univoco, utili sia per la detection che per il tracking.

Le annotazioni contengono:

- ID dell'oggetto;
- coordinate della bounding box;
- visibilità (tasso di occlusione);
- classe dell'oggetto.

Sono state selezionate esclusivamente le cartelle relative a FRCNN, poiché esse presentano annotazioni direttamente distinguibili per classe. Inoltre, sono stati filtrati i bounding box con classi non pertinenti. In particolare, sono state mantenute solo le classi: **1** (Pedestrian), **2** (Person on vehicle), **7** (Static person), tutte riconducibili alla presenza umana. I bounding box totalmente esterni all'immagine sono stati eliminati, mentre quelli parzialmente fuori sono stati ridimensionati affinché rientrassero interamente nell'immagine.

Table 3.1: Statistiche dei video utilizzati dal dataset MOT17-FRCNN per il training

Nome	FPS	Risoluzione	Durata (frame)	Box totali	Densità	Descrizione
MOT17-13-FRCNN	25	1920×1080	750	11642	15.5	Filmato da un autobus in un incrocio affollato
MOT17-11-FRCNN	30	1920×1080	900	9436	10.5	Telecamera in movimento in un centro commerciale
MOT17-10-FRCNN	30	1920×1080	654	12839	19.6	Scena pedonale notturna, camera in movimento
MOT17-09-FRCNN	30	1920×1080	525	5325	10.1	Scena urbana pedonale, angolo basso
MOT17-05-FRCNN	14	640×480	837	6917	8.3	Scena urbana da piattaforma mobile
MOT17-04-FRCNN	30	1920×1080	1050	47557	45.3	Scena notturna pedonale, punto di vista elevato
MOT17-02-FRCNN	30	1920×1080	600	18581	31.0	Persone in una grande piazza
Totale	-	-	5316	122297	23.0	-

3.1.2 MOT20

MOT20 [1] è un'estensione focalizzata su scene ad alta densità pedonale. Contiene 4 sequenze ad alta risoluzione per un totale di circa 13.000 frame annotati. In alcuni frame sono presenti fino a 246 persone visibili simultaneamente. Sono state mantenute solo le classi umane sopra menzionate

e sono state applicate le stesse regole di ridimensionamento e filtraggio dei box di MOT17. Per quanto riguarda il dataset MOT20, sono stati utilizzati i seguenti video nella fase di training. La Tabella 3.2 riassume le caratteristiche principali, inclusi risoluzione, durata, numero totale di annotazioni e densità media di persone per frame. Gli script per entrambi i dataset MOT si trovano in Appendice C

Table 3.2: Statistiche dei video utilizzati dal dataset MOT20 per il training

Nome	FPS	Risoluzione	Durata (frame)	Box totali	Densità	Descrizione
MOT20-05	25	1654×1080	3315	751330	226.6	Piazza affollata di notte
MOT20-03	25	1173×880	2405	356728	148.3	Ingresso stadio, punto di vista elevato
MOT20-02	25	1920×1080	2782	202215	72.7	Stazione ferroviaria affollata (indoor)
MOT20-01	25	1920×1080	429	26647	62.1	Stazione ferroviaria affollata (indoor)
Totale	-	-	8931	1336920	149.7	-

3.1.3 CrowdHuman

Il dataset **CrowdHuman** [2] è stato incluso per aumentare la varietà e la complessità delle scene. Ogni immagine ad alta risoluzione presenta in media 22 persone, spesso occluse o sovrapposte. Sono state utilizzate esclusivamente le bounding box `full_body`, considerate rappresentative dell'intera persona. I box con punto iniziale o finale fuori dall'immagine sono stati ridimensionati per rientrare completamente nel frame.

Per la conversione delle annotazioni in formato compatibile con YOLO, è stato utilizzato uno script Python che effettua la normalizzazione e la correzione dei box. Lo script completo è riportato in Appendice A

Formato delle annotazioni Ogni persona è annotata con tre tipi di bounding box:

- `full_body`: corpo intero (usato nel progetto);
- `visible_body`: solo la parte visibile;
- `head_box`: testa.

Nel contesto del presente lavoro, si è scelto di mantenere le annotazioni di tipo `full_body`, poiché l'obiettivo è quello di applicare il sistema a scene

mediamente affollate, in cui le persone sono generalmente distinguibili e ben separate. In scenari estremamente congestionati, come quelli tipici del *crowd counting*, sarebbe invece preferibile utilizzare annotazioni di tipo **head_box**, in quanto la testa rappresenta spesso l'unica parte visibile dell'individuo. Tale scelta consente una maggiore robustezza nella rilevazione in condizioni di forte sovrapposizione tra soggetti, ma risulta meno adatta per applicazioni che richiedono il rilevamento del corpo intero, come nel nostro caso.

Nome	Risoluzione	Box totali	Densità	Descrizione
CrowdHuman	Variabile	339 565	22.6	Dataset di immagini in ambienti urbani affollati, con annotazioni dettagliate e viste eterogenee.

Table 3.3: Statistiche del dataset CrowdHuman utilizzato per il training.

3.1.4 CUHK-Pedestrian

Il dataset **CUHK-Pedestrian** [3] è stato incluso per aggiungere scene urbane aggiuntive. I file originali erano in formato **.seq** e sono stati convertiti in **.mp4** tramite tool esterno, per poter estrarre i frame da utilizzare. Le annotazioni contenute nei file **.vbb** sono state convertite in formato YOLO tramite uno script Python personalizzato che si trova in Appendice B

3.1.5 Crowd TypSA (Escluso)

Il dataset **Crowd TypSA** [4] è stato inizialmente considerato per l'inclusione nel training. Tuttavia, è stato successivamente escluso a causa dell'elevato rumore nelle annotazioni. Nonostante il tentativo di rimuovere bounding box palesemente errati, il modello peggiorava nelle performance. Si sospetta che le annotazioni siano state generate automaticamente da una rete neurale, risultando quindi poco affidabili.

3.2 Preprocessing e Conversione

3.2.1 Formato delle Annotazioni in YOLO12

YOLO12 richiede bounding box nel formato seguente:

$$\langle class \rangle \quad \langle x_center \rangle \quad \langle y_center \rangle \quad \langle width \rangle \quad \langle height \rangle$$

Dove:

- **class**: intero che rappresenta la classe dell'oggetto;
- **x_center, y_center**: coordinate normalizzate del centro della bounding box;
- **width, height**: larghezza e altezza normalizzate rispetto alla dimensione dell'immagine.

Nei dataset originali, le bounding box sono definite a partire dal vertice in alto a sinistra. La conversione è avvenuta calcolando il centro delle box e normalizzando i valori in base alle dimensioni del frame. Inoltre, tutte le box con punto iniziale o finale fuori dall'immagine sono state corrette per rientrare nei limiti del frame.

3.3 Distribuzione spaziale delle annotazioni

Per analizzare la distribuzione spaziale delle annotazioni nel training set, sono state generate delle heatmap che evidenziano le regioni più frequentemente occupate dai *bounding box* delle persone. Lo script completo è riportato in Appendice D

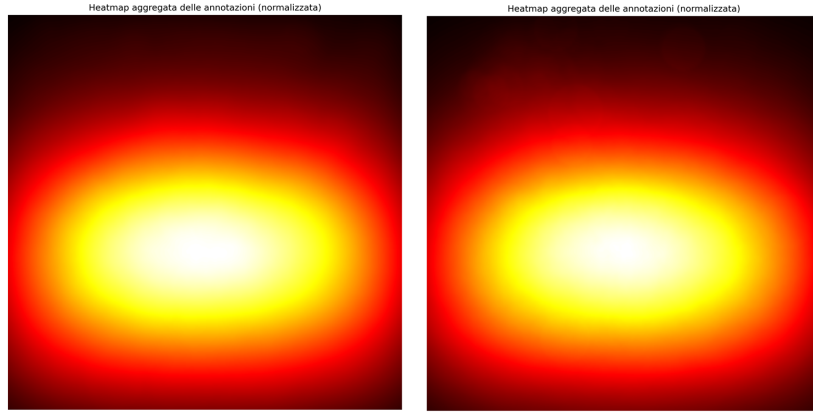


Figure 3.1: Heatmap aggregata delle annotazioni nel dataset finale. A sinistra la heatmap relativa al training set e a destra quella relativa al validation set. Le aree più chiare indicano le zone in cui i bounding box sono più frequenti. L’analisi rivela una concentrazione centrale, coerente con la disposizione tipica delle persone nei video di sorveglianza o in ambienti urbani.

L’obiettivo principale della generazione delle heatmap non è solo descrittivo, ma anche funzionale alla valutazione della distribuzione spaziale delle annotazioni nei due subset. In particolare, è fondamentale verificare che il validation set presenti una distribuzione coerente con quella del training set, evitando così forme di *overfitting spaziale*.

Infatti, se durante il training il modello fosse esposto principalmente a persone localizzate in una determinata regione dell’immagine (ad esempio al centro), potrebbe sviluppare una forte dipendenza da tale configurazione. Di conseguenza, qualora nel validation set le annotazioni fossero concentrate in regioni differenti (ad esempio spostate verso i bordi), si rischierebbe una significativa perdita di performance.

Nel nostro caso, infatti, le heatmap mostrano una distribuzione coerente tra training e validation, confermando che entrambi i set seguono una disposizione rappresentativa della realtà. Sottolineiamo che la coerenza tra le due heatmap è assolutamente normale, seppur il validation sia stato scelto oculatamente per garantire varietà nelle scene. Questo perchè la distribuzione spaziale è maggiormente legata al tipo di problema che siamo affrontando. Difatti è perfettamente normale e comprensibile il motivo per cui la maggior parte delle bounding box ricadano al centro dell’immagine.

3.3.1 Sampling dei Frame

Per costruire il dataset finale a partire da MOT17 e MOT20, sono stati estratti **due frame al secondo** da ciascun video. Questo approccio riduce la ridondanza e migliora la varietà del training set, evitando che immagini troppo simili appesantiscano l'apprendimento.

3.3.2 Struttura delle Cartelle e Validazione

Il dataset è organizzato secondo una struttura compatibile con YOLO12. La seguente rappresentazione ad albero mostra l'organizzazione delle cartelle del dataset all'interno della directory principale:

```
dataset/  
|-- images/  
|   |-- train/  
|   |-- val/  
|   |-- test/  
|-- labels/  
|   |-- train/  
|   |-- val/  
|   |-- test/  
|-- data.yaml
```

La struttura è suddivisa in due componenti principali:

- **images/**: contiene le immagini effettive utilizzate durante la fase di addestramento e validazione, suddivise in **train/**, **val/** e **test/**.
- **labels/**: contiene i file di annotazione in formato YOLO, ciascuno in corrispondenza a un'immagine. Ogni file ha lo stesso nome del file immagine associato, ma con estensione **.txt**.
- **data.yaml**: è un file di configurazione che specifica il percorso delle immagini per training e validation, il numero di classi (**nc**) e la lista delle classi da rilevare.

Ogni file di annotazione in `labels/` segue il formato YOLO, in cui ogni riga rappresenta un oggetto rilevato:

```
<class_id> <x_center> <y_center> <width> <height>
```

Tutti i valori sono normalizzati tra 0 e 1 rispetto alla dimensione dell'immagine. Ad esempio:

```
0 0.521 0.674 0.093 0.210
```

Questa riga indica una persona (classe 0) con un bounding box centrato al 52.1% della larghezza e 67.4% dell'altezza dell'immagine, con larghezza pari al 9.3% e altezza pari al 21.0% dell'immagine stessa.

Il file `data.yaml` associato al dataset ha una struttura del tipo:

```
train: dataset/images/train
val: dataset/images/val

nc: 1
names: ['Persona']
```

Questa configurazione permette al framework YOLO di caricare correttamente il dataset, gestire le annotazioni e associare ogni classe al suo nome corrispondente.

Un'organizzazione strutturata e coerente del dataset è fondamentale per il corretto funzionamento dei processi di addestramento, validazione e inferenza. Inoltre, una separazione chiara tra immagini e annotazioni consente di gestire più facilmente task di preprocessing, statistiche sui dati e analisi qualitative delle predizioni.

3.4 Validation set

La suddivisione del dataset in training e validation set è stata eseguita sia in modo automatico che manuale, a seconda delle caratteristiche del dataset di origine.

Per il dataset **CrowdHuman**, è stato rispettato lo *split* ufficiale fornito dagli autori, che garantisce una distribuzione bilanciata delle scene e delle annotazioni.

Per quanto riguarda i dataset **MOT17** e **CUHK-Pedestrian**, la suddivisione è stata effettuata manualmente, selezionando circa il 20% dei video complessivi per il validation set. In particolare, sono stati scelti 2 video su 11 totali per il MOT17, con l'obiettivo di mantenere la varietà in termini di contesto, densità di persone, angolazioni e condizioni di acquisizione. Un criterio analogo è stato adottato anche per CUHK-Pedestrian.

È importante sottolineare che i dati selezionati manualmente costituiscono una porzione ridotta del dataset complessivo, ma sono stati scelti con attenzione per assicurare al validation set una sufficiente eterogeneità. Questa eterogeneità è fondamentale per valutare in modo realistico la capacità del modello di generalizzare a scenari diversi da quelli visti in fase di addestramento.

3.5 Test Set

Il test set utilizzato per la valutazione finale del modello è stato acquisito direttamente da noi, al fine di disporre di un insieme di dati completamente indipendente dai dataset utilizzati per l'addestramento e la validazione.

L'acquisizione è stata effettuata registrando video e catturando immagini in una varietà di contesti reali, al fine di ottenere un campione quanto più eterogeneo possibile. Le riprese sono state effettuate in condizioni di luce diverse (giorno, sera, ambienti artificialmente illuminati), in luoghi differenti (spazi aperti, strade urbane, eventi), e in situazioni con livelli di affollamento variabili, includendo sia scene con bassa densità di persone che situazioni particolarmente congestionate.

Questa scelta metodologica è stata adottata per garantire una valutazione affidabile e realistica delle capacità del modello, simulando le condizioni d'uso più vicine a quelle operative. L'eterogeneità del test set rappresenta infatti un elemento chiave per misurare la robustezza e la generalizzazione del modello su scenari mai visti prima.

Tutte le annotazioni sono state effettuate manualmente utilizzando la pi-

attaforma **Roboflow**, e la piattaforma **CBAT** seguendo criteri rigorosi di precisione e coerenza. Si è cercato di etichettare con attenzione ogni singola immagine, marcando esclusivamente le persone visibili e distinguibili, e assicurandosi che i bounding box rispettassero la reale estensione dell'individuo nella scena. In presenza di casi ambigui (es. parziali occlusioni o soggetti molto lontani), è stata adottata una linea guida condivisa per mantenere la coerenza dell'intero set.

Difatti, l'utilizzo di un test set indipendente, etichettato con cura e composto da scene realistiche e non artificiali, permette di valutare le reali capacità del modello in condizioni operative, superando i limiti di valutazioni condotte su dati sintetici o troppo omogenei. Questo approccio rafforza l'affidabilità delle metriche di performance e consente di individuare eventuali punti deboli del modello in termini di adattabilità a contesti non visti. Risulta quindi una parte cruciale del progetto.

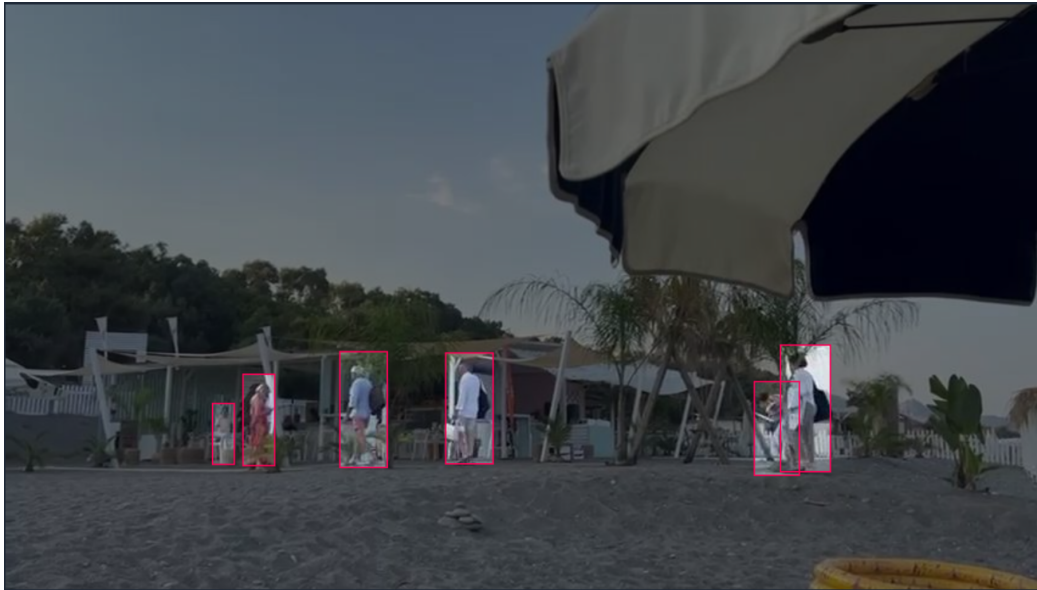


Figure 3.2: Frame prelevato da un video del test set.

3.6 Statistiche del Dataset di Test

Di seguito si riportano le principali statistiche:

Table 3.4: Statistiche del dataset di test

Numero di immagini	144
Numero totale di annotazioni	1671
Media annotazioni per immagine	11.6
Risoluzione media (MP)	5.99
Risoluzione minima	0.41 MP
Risoluzione massima	16.05 MP
Rapporto d'aspetto mediano	3264x2048
Classi presenti	Persona

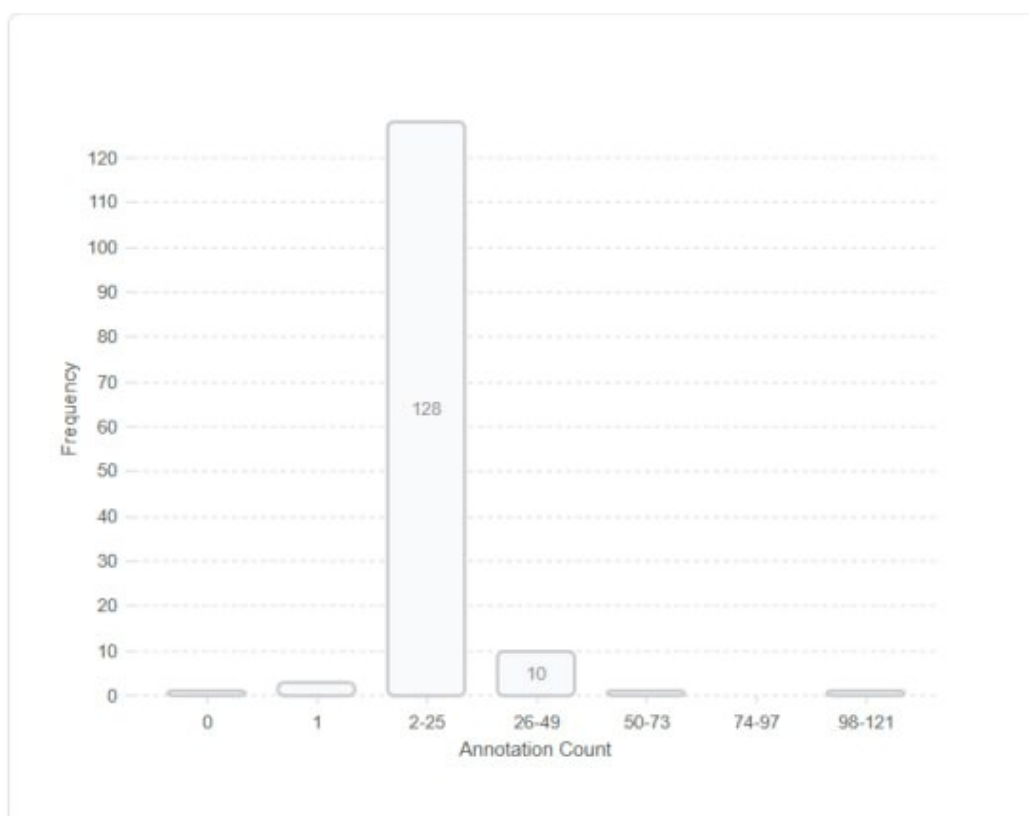


Figure 3.3: Annotation count.

L'analisi del dataset mostra che la maggior parte delle immagini contiene un numero moderato di persone (tra 2 e 25), ma sono presenti anche immagini con alta densità di annotazioni, alcune delle quali superano i 100 individui. Questo permette di testare il modello su casi semplici e complessi, verifican-

done la capacità di generalizzare in presenza sia di scene poco affollate che di ambienti altamente congestionati.

La variabilità nelle dimensioni e nei rapporti d'aspetto delle immagini introduce un ulteriore livello di difficoltà, rendendo il test particolarmente indicativo per valutare la robustezza del modello.

Chapter 4

Metodologia

In questo capitolo vengono descritte nel dettaglio le metodologie adottate per lo sviluppo del progetto, a partire dalle operazioni preliminari di pre-processing dei dati fino alle strategie utilizzate per migliorare le prestazioni del modello.

L'architettura utilizzata per il training è basata su **YOLO12**, descritta nel Capitolo 2, e qui si farà riferimento soltanto agli aspetti implementativi e alla parametrizzazione adottata durante l'addestramento.

L'obiettivo è fornire una visione completa e trasparente del flusso di lavoro che ha guidato la realizzazione del sistema di rilevamento e conteggio delle persone.

Sfide principali nel nostro progetto

Le principali difficoltà dell'object detection che abbiamo riscontrato durante lo sviluppo del progetto sono le seguenti:

- **Presenza di persone parzialmente occluse**, che rende difficile identificare contorni completi;
- **Dimensione variabile degli oggetti**, dovuta alla distanza rispetto alla camera;
- **Persone molto piccole e lontane**: per motivi computazionali, le immagini in input sono state ridimensionate a 640×640 , sacrificando

parte della risoluzione. Questo compromesso è necessario per mantenere tempi di training accettabili, ma rende difficile la rilevazione di soggetti lontani che occupano pochissimi pixel;

- **Variazioni di illuminazione e prospettiva** nei frame reali;
- **Falsi positivi dovuti a oggetti simili** alla forma umana, come sagome, poster o manichini;
- **Costruzione Dataset di Training:** per ottenere buone prestazioni, è stato necessario costruire un dataset di addestramento che includesse scene varie (urbane, stazionarie, in movimento) e diverse condizioni di illuminazione e densità di folla. Questa varietà è fondamentale per garantire la generalizzazione del modello a contesti non visti;
- **Difficoltà di reperimento di dataset ben etichettati:** molti dataset pubblici presentano annotazioni rumorose, incomplete o non uniformi. È stato quindi necessario selezionare con attenzione le fonti (MOT17, MOT20, CrowdHuman) ed effettuare un preprocessing accurato per ottenere un insieme di dati affidabile per l'addestramento; inoltre, i dataset dello stato dell'arte sono difficilmente reperibili, al contrario di quanto si possa immaginare.

Il ridimensionamento delle immagini a 640×640 è stato necessario per rendere il training del modello YOLO12 più veloce e gestibile anche su hardware non estremamente potente; inoltre YOLO12 permette di gestirlo implicitamente. Tuttavia, questo ha comportato una perdita di dettaglio nelle aree più lontane dell'immagine, dove le persone possono occupare porzioni minime del frame e risultare difficili da distinguere dallo sfondo. Nelle sezioni successive parleremo di come abbiamo cercato di mitigare il problema.

4.1 Preprocessing dei dati

Il dataset utilizzato per l'addestramento del modello è composto dai benchmark pubblici **MOT17** e **MOT20**, i quali forniscono annotazioni in formato

`gt.txt` secondo lo standard della Multiple Object Tracking Challenge. Tuttavia, questo formato non è direttamente compatibile con quello richiesto da YOLO, pertanto è stato necessario implementare una fase di preprocessing per convertirlo nel formato corretto.

In particolare, ciascun file `gt.txt` rappresenta tutte le annotazioni di un video, con bounding box espresse in coordinate assolute rispetto al frame, e include molteplici classi di oggetti. Per renderlo utilizzabile da YOLO, è stato necessario:

- suddividere il file `gt.txt` in N **file distinti**, uno per ciascun frame del video, dove N è il numero totale di frame;
- convertire le coordinate delle bounding box nel formato richiesto da YOLO, ovvero (`classe`, `x_center`, `y_center`, `width`, `height`), normalizzate rispetto alle dimensioni del frame;
- ridimensionare o correggere le bounding box che fuoriuscivano dai limiti del frame;
- filtrare le annotazioni mantenendo **solo le tre classi** che rappresentano le persone, ignorando tutte le altre categorie;
- unificare le tre classi relative alle persone in un'unica classe denominata **Person**.

Tutte queste operazioni sono state realizzate tramite uno script Python, che ha anche provveduto a ricostruire la struttura delle cartelle necessaria per l'utilizzo del dataset da parte del framework YOLO.

Una volta generati i file di annotazione nel formato corretto, è stato creato un file di configurazione in formato `.yaml`, contenente:

- il percorso alle immagini di addestramento e validazione;
- il numero di classi (in questo caso 1);
- il nome della classe (**Person**).

Infine, ai fini dell'addestramento sono stati selezionati solo i video etichettati come **FRCNN** all'interno dei dataset MOT17 e MOT20, in quanto forniscono un livello di accuratezza più elevato nelle annotazioni rispetto ad altri tracker automatici presenti nella challenge.

4.2 Training e Setup Sperimentale

L'addestramento dei modelli YOLO12 (nelle varianti YOLO12-n, YOLO12-s, YOLO12-m) è stato eseguito utilizzando i parametri standard raccomandati dalla documentazione ufficiale Ultralytics, in modo da mantenere un comportamento il più aderente possibile agli scenari sperimentali previsti.

4.2.1 Configurazione dell'Addestramento

Per garantire un buon compromesso tra tempo di convergenza, stabilità dell'apprendimento e prestazioni finali, sono stati adottati i seguenti valori:

- **Numero di epoche:** 100
- **Batch size:** 12 o 16 in base al modello (in linea con la capacità della GPU utilizzata).
- **Learning rate iniziale:** 0.01
- **Scheduler:** Cosine Annealing con warm-up iniziale
- **Ottimizzatore:** Stochastic Gradient Descent (SGD) con momentum

L'addestramento è stato gestito attraverso appositi file di configurazione `.yaml`, contenenti:

- percorsi per immagini e etichette per training e validation;
- nome e numero delle classi (una sola classe: **Person**);

4.2.2 Dettagli dell'Ottimizzazione

L'ottimizzatore scelto è stato **SGD**, in accordo con il comportamento predefinito di YOLO12, configurato con:

- `optimizer = SGD`
- `lr0 = 0.01`
- `lrf = 0.01`
- `momentum = 0.937`
- `weight_decay = 1e-5`
- `warmup_epochs = 3`
- `warmup_momentum = 0.8`
- `cos_lr = True` (cosine decay del learning rate)

Il learning rate decresce secondo una curva cosenoidale fino a raggiungere un valore minimo predefinito, permettendo una fine regolazione dei pesi nelle fasi finali di training.

Ci sono altri iper-parametri lasciati di default che non citiamo per brevità, tra cui quelli che gestiscono la data augmentation. In ogni caso tutti gli iper-parametri si trovano nel file `args.yaml` contenuto dentro la repository principale, nel percorso `training_result/nome_modello`

4.2.3 Ambiente di Esecuzione

Il training è stato svolto principalmente in locale, su una GPU **NVIDIA RTX 4070 SUPER da 12GB**, con supporto CUDA abilitato. In alcuni casi, per esperimenti esplorativi e debugging, è stata utilizzata la piattaforma **Google Colab Pro+**, con salvataggio periodico dei checkpoint.

4.2.4 Dataset e Considerazioni Pratiche

Durante l'addestramento sono stati utilizzati dataset ad alta densità come **MOT17**, **MOT20**, **CrowdHuman** e **CUHK-Pedestrian**, i quali presentano scene complesse e numerosi oggetti ravvicinati. Tali condizioni richiedono un apprendimento robusto per evitare overfitting e migliorare la generalizzazione.

4.2.5 Strategie di generalizzazione

Per ridurre il rischio di overfitting e migliorare la capacità del modello di generalizzare a dati non visti, sono state adottate diverse strategie:

- **Data Augmentation:** sono stati applicati random flip, scaling, cropping, blur, e variazioni di illuminazione, in modo da aumentare la variabilità dei dati di input e simulare contesti realistici.
- **Validazione su dati eterogenei:** il validation set è stato costruito appositamente in modo da presentare una distribuzione spaziale e ambientale differente rispetto al training set, per testare la robustezza del modello in condizioni variabili.
- **Soglia di confidenza dinamica:** nella fase di validazione sono state testate diverse soglie di confidenza per valutare l'impatto sui falsi positivi e falsi negativi.
- **Early-Stopping:** ovvero il training poteva potenzialmente terminare prematuramente qualora la loss sul validation rimanesse quasi costante per un numero fissato di epoche, evitando così il sovradattamento del modello.
- **Salvataggio dei pesi migliori:** è stato utilizzato un meccanismo di salvataggio automatico del modello che otteneva la miglior **mAP50-95** sul validation set.
- **Bilanciamento delle annotazioni:** tutte le annotazioni relative alle classi non rilevanti sono state eliminate in fase di preprocessing, mante-

nendo una singola classe target e garantendo un bilanciamento costante tra i frame.

Tali strategie hanno permesso di migliorare le prestazioni del modello sui dati di test e aumentare l'affidabilità dei risultati ottenuti in fase di valutazione.

4.2.6 Utilizzo di SAHI (Slicing Aided Hyper Inference)

Nel contesto del progetto è stato integrato **SAHI (Slicing Aided Hyper Inference)** [5], un framework open-source progettato per migliorare le prestazioni dei modelli di object detection in presenza di **oggetti piccoli, parzialmente occlusi o distanti nella scena**, come accade frequentemente in contesti di sorveglianza o crowd analysis.

Cos'è SAHI

SAHI è un metodo di *inference slicing*, che divide l'immagine ad alta risoluzione in più sotto-regioni (slice) sovrapposte. Ogni slice viene processata indipendentemente da un detector pre-addestrato (es. YOLO12). Le predizioni vengono poi fuse in un'unica mappa tramite **Non-Maximum Suppression (NMS)** globale. Questa strategia consente di **preservare la risoluzione locale** degli oggetti piccoli, migliorandone la rilevazione.

Non-Maximum Suppression (NMS). La *Non-Maximum Suppression* (NMS) è una tecnica fondamentale nei moderni algoritmi di *object detection*, impiegata per filtrare le predizioni multiple che si riferiscono allo stesso oggetto. Durante l'inferenza, un detector come YOLO può generare molteplici bounding box sovrapposte che predicono la stessa classe per uno stesso oggetto. In assenza di un meccanismo di soppressione, queste predizioni ridondanti comprometterebbero l'accuratezza del modello, generando falsi positivi o conteggi errati.

La NMS opera selezionando, tra le bounding box che predicono la stessa classe, quella con il punteggio di confidenza più elevato. Tutte le altre box che presentano con essa un'*Intersection over Union (IoU)* superiore a una soglia predefinita vengono eliminate. Tale procedura viene ripetuta iterativamente

per tutte le classi e per tutte le box rilevate, fino a ottenere una sola predizione per ciascun oggetto. Formalmente, data una lista di box $B = \{b_1, b_2, \dots, b_n\}$ ordinate per confidenza decrescente, la NMS seleziona b_1 e rimuove da B tutte le box con $\text{IoU}(b_1, b_i) > \tau$, con τ soglia prefissata, e continua il processo con i rimanenti elementi.

Questa procedura consente di mantenere la box più rappresentativa per ciascun oggetto, riducendo la sovrapposizione e migliorando la leggibilità e l'affidabilità delle predizioni finali. In combinazione con tecniche di slicing come SAHI, la NMS viene applicata a livello globale per unificare le predizioni provenienti da regioni parziali dell'immagine, contribuendo a preservare l'integrità semantica del risultato.

Motivazione e vantaggi

Secondo il paper originale [5], l'approccio SAHI:

- è compatibile con qualsiasi modello pre-addestrato;
- migliora la detection AP (*average precision*) fino a **+6.8%** usando solo slicing in inferenza;
- se combinato con fine-tuning sui patch, arriva fino a **+14.5%** AP su oggetti piccoli;
- può essere integrato facilmente senza aumentare l'uso di memoria GPU (slicing parallelo);

Parametri di Inferenza con SAHI

L'inferenza è stata eseguita con i seguenti parametri:

- **slice_height** = 640 Altezza di ciascuna porzione (in pixel).
- **slice_width** = 640 Larghezza di ciascuna porzione (in pixel).
- **overlap_height_ratio** = 0.2 Sovrapposizione verticale tra slice adiacenti (20%).

- **overlap_width_ratio** = 0.2 Sovrapposizione orizzontale tra slice adiacenti (20%).
- **confidence_threshold** = 0.35 Soglia di confidenza minima per accettare una predizione. La soglia scelta è quella che massimizza la generalizzazione secondo le curve illustrate nel capitolo dei risultati sperimentali.
- **perform_standard_pred** = False Disattiva l'inferenza sull'intera immagine.
- **device** = 0 Utilizzo della GPU (primo dispositivo CUDA disponibile).

L'inferenza è stata dunque eseguita esclusivamente sulle slice, evitando la predizione sull'immagine completa. Questa strategia consente di migliorare significativamente la rilevazione di oggetti piccoli situati ai bordi o in zone ad alta densità, grazie alla maggiore risoluzione delle porzioni analizzate.

SAHI e Risultati

L'utilizzo di SAHI ha permesso di aumentare la qualità delle rilevazioni nei nostri video, riducendo il numero di falsi negativi su persone distanti o parzialmente visibili. L'implementazione si è rivelata efficace e scalabile, migliorando la robustezza del sistema in condizioni reali.

Chapter 5

Metriche di valutazione per l'Object Detection

Per valutare le prestazioni di un modello di *object detection*, è fondamentale disporre di metriche quantitative che tengano conto sia della capacità del modello di localizzare correttamente gli oggetti nelle immagini, sia della sua precisione nel classificarli. In questa sezione vengono presentate le principali metriche utilizzate: **Precision**, **Recall**, **IoU**, **mAP** e **F1-score**.

5.1 Intersection over Union (IoU)

L'**Intersection over Union (IoU)** misura la sovrapposizione tra la bounding box predetta (B_p) e quella reale (B_{gt}). È definita come:

$$IoU = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|} \quad (5.1)$$

Il valore di IoU varia tra 0 e 1, dove 1 indica una corrispondenza perfetta. Una predizione è considerata corretta se l'IoU supera una certa soglia, comunemente fissata a 0.5 (detta anche *IoU threshold*).

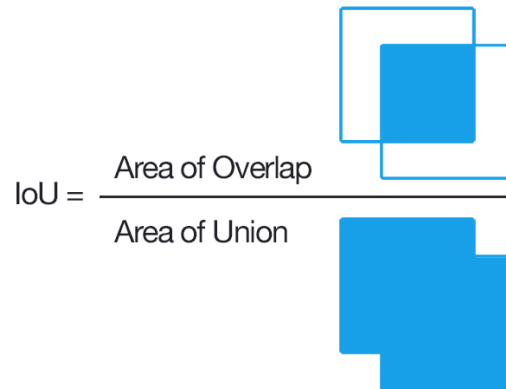


Figure 5.1: Visualizzazione del calcolo dell'IoU: rapporto tra area di sovrapposizione e area di unione.

5.2 Precisione e Richiamo (Precision & Recall)

- **Precision** (*precisione*): la frazione di predizioni corrette tra tutte quelle effettuate dal modello.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** (*richiamo*): la frazione di oggetti rilevati correttamente rispetto al totale degli oggetti presenti.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Dove:

- TP = True Positives (predizioni corrette),
- FP = False Positives (predizioni errate),
- FN = False Negatives (oggetti mancati).

Nota sull'influenza della soglia di decisione:

Nella pratica, i modelli di classificazione non restituiscono direttamente etichette discrete (*persona*, *sfondo*, ecc.), bensì un valore di **confidenza** (ovvero una

probabilità) per ciascuna classe. Affinché una predizione venga considerata positiva, è necessario che la confidenza superi una soglia predefinita τ , compresa tra 0 e 1. Di conseguenza, il calcolo delle metriche come *Precision* e *Recall* dipende direttamente dal valore di τ scelto.

- Se τ è troppo basso, il modello effettuerà molte predizioni positive, aumentando i *False Positives* (FP) e quindi abbassando la *Precision*.
- Se τ è troppo alto, il modello sarà molto selettivo, rischiando di perdere oggetti realmente presenti, aumentando i *False Negatives* (FN) e quindi abbassando il *Recall*.

Nel contesto dell'**object detection**, oltre alla soglia di confidenza τ viene anche introdotta una soglia sull'IoU (Intersection over Union), indicata come θ_{IoU} , per stabilire se una predizione sia sufficientemente vicina a un oggetto reale. Una predizione viene considerata corretta (*True Positive*) solo se:

$$\text{confidenza} \geq \tau \quad \text{e} \quad \text{IoU} \geq \theta_{IoU}$$

Per questo motivo, nella valutazione di modelli di object detection si fa spesso riferimento a *curve di precisione e richiamo* al variare della soglia τ , oppure a metriche aggregate come la **mAP** (mean Average Precision), che tiene conto delle prestazioni su più soglie e valori di IoU.

5.3 F1-score

Il **F1-score** è la media armonica tra precision e recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.2)$$

Fornisce una misura bilanciata tra falsi positivi e falsi negativi, utile in caso di squilibrio tra le classi o dataset con molte difficoltà.

5.4 Average Precision e Mean Average Precision

La **Average Precision (AP)** è una metrica che misura l'area sotto la curva *Precision-Recall* per una determinata classe. Essa tiene conto sia della ca-

pacità del modello di identificare correttamente gli oggetti (precisione), sia della sua capacità di trovarli tutti (richiamo). L'AP è calcolata come segue:

$$AP = \int_0^1 p(r) dr \quad (5.3)$$

dove $p(r)$ è la funzione di precisione in funzione del recall r . Nella pratica, questa formula viene approssimata con una somma discreta:

$$AP = \sum_{n=1}^N (R_n - R_{n-1}) \cdot P_n \quad (5.4)$$

dove:

- P_n è la precisione al punto n ,
- R_n è il valore di recall al punto n ,
- N è il numero totale di soglie considerate,
- $(R_n - R_{n-1})$ rappresenta il passo nel recall.

AP rappresenta dunque una misura aggregata delle prestazioni del modello su una singola classe.

La **mean Average Precision (mAP)** è la media delle AP su tutte le classi considerate:

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c \quad (5.5)$$

dove:

- C è il numero totale di classi,
- AP_c è l'Average Precision della classe c .

5.4.1 Considerazioni sul ruolo della soglia IoU in Object Detection

Nel contesto della *object detection*, il calcolo della **Average Precision (AP)** e, di conseguenza, della **mean Average Precision (mAP)**, si basa non soltanto sulle metriche tradizionali di precisione e richiamo, ma anche sull'uso della **IoU (Intersection over Union)** come criterio di corrispondenza tra le predizioni del modello e i ground truth. Tale valore, come detto precedentemente, è compreso tra 0 e 1 e misura quanto due box si sovrappongono. Più è alto, più i due box coincidono.

Ruolo della soglia IoU

Nel calcolo della AP, una predizione è considerata **true positive** solo se:

- la classe è corretta;
- la IoU tra il box predetto e quello reale supera una **soglia predefinita**, ad esempio 0.5.

Questa soglia può essere fissata a un singolo valore (es. 0.5), oppure può variare in un intervallo.

mAP@0.5 e mAP@0.5:0.95

Le due principali varianti del *mean Average Precision* sono:

- **mAP@0.5**: si calcola la AP per ciascuna classe utilizzando come soglia IoU il valore fisso di 0.5.
- **mAP@0.5:0.95**: definita nello standard COCO, questa metrica calcola la media delle AP per ciascuna classe su 10 soglie di IoU comprese tra 0.5 e 0.95 con step di 0.05, ovvero:

$$\text{mAP@0.5:0.95} = \frac{1}{10} \sum_{t \in \{0.5, 0.55, \dots, 0.95\}} \text{AP}_{\text{IoU}=t}$$

È considerata più rigorosa e realistica in quanto valuta il comportamento del modello su diverse soglie di matching.

Nota sul termine mAP

Quando si parla semplicemente di “mAP” senza specificare una soglia, ci si riferisce per convenzione a **mAP@0.5:0.95**. Tuttavia, è buona norma indicare sempre esplicitamente la soglia o l'intervallo utilizzato per evitare ambiguità.

Differenza con la Mean Precision (mp)

Alcuni framework, come YOLO12, forniscono anche il valore di **Mean Precision (mp)**, calcolato come media delle precisioni sulle classi a una soglia fissa (da cercare nella documentazione). A differenza della AP, la mp **non tiene conto della variazione della soglia di confidenza né dell'IoU**, risultando pertanto meno informativa e meno standardizzata in object detection.

5.5 Loss functions

In questa sezione vengono illustrate le principali funzioni di *loss* utilizzate nel modello. Tali funzioni sono monitorate durante entrambe le fasi di *training* e *validation*, e forniscono indicazioni utili sull'andamento dell'apprendimento del modello.

Le principali funzioni di loss sono le seguenti:

- **train/box_loss**: rappresenta la perdita associata alla localizzazione dei bounding box durante il training. Una box_loss elevata indica che il modello fatica a predire correttamente la posizione degli oggetti.
- **train/cls_loss**: è la perdita legata alla classificazione degli oggetti rilevati. Valori elevati di questa loss suggeriscono che il modello commette errori nel riconoscere la classe di appartenenza degli oggetti durante l'addestramento.
- **train/df_l_loss**: corrisponde alla *Distribution Focal Loss*, una funzione pensata per dare maggiore peso alle predizioni errate relative a istanze difficili o meno rappresentate nel dataset. È particolarmente utile per gestire squilibri nei dati e migliorare la precisione su esempi complessi.

- **val/box_loss**: analoga alla **train/box_loss**, ma calcolata sul validation set. Serve a valutare se il modello sta imparando a localizzare correttamente anche su dati non visti.
- **val/cls_loss**: versione di validazione della **train/cls_loss**. Permette di monitorare la capacità del modello di generalizzare nella classificazione.
- **val/dfl_loss**: è la **Distribution Focal Loss** calcolata sul validation set. Aiuta a comprendere se il modello mantiene buone prestazioni anche su esempi complessi non appartenenti al training set.

5.6 Matrice di Confusione

La matrice di confusione è uno strumento fondamentale per valutare le prestazioni di un classificatore, in particolare nei problemi di classificazione binaria. Essa mostra come le predizioni del modello si distribuiscono rispetto alle etichette reali, consentendo di distinguere tra predizioni corrette e errori di classificazione.

Nel caso generale, un classificatore binario può emettere due classi: **Positiva** e **Negativa**. La matrice di confusione associata può essere rappresentata come segue:

Reale / Predetto	Positiva	Negativa
Positiva	True Positive (TP)	False Negative (FN)
Negativa	False Positive (FP)	True Negative (TN)

Table 5.1: Struttura standard di una matrice di confusione binaria.

- **True Positive (TP)**: il modello predice correttamente la classe positiva quando essa è realmente presente.
- **False Positive (FP)**: il modello predice la classe positiva quando invece la classe reale è negativa (falso allarme).
- **False Negative (FN)**: il modello predice la classe negativa quando in realtà è presente la classe positiva (mancata rilevazione).

- **True Negative (TN)**: il modello predice correttamente la classe negativa quando essa è effettivamente presente.

Essa quindi rappresenta una struttura sintetica che raccoglie, in forma tabellare, tutte le informazioni essenziali (TP, FP, FN, TN) relative alle prestazioni di un classificatore. Da essa è possibile derivare agevolmente metriche già descritte in precedenza, come *accuracy*, *precision*, *recall* e *F1-score*.

Nei contesti multi-classe, la matrice viene estesa in dimensione per includere tutte le classi previste dal modello, mantenendo invariata la logica di interpretazione.

Influenza della soglia decisionale. È importante sottolineare che la costruzione della matrice di confusione dipende da una **soglia decisionale**, ovvero un valore di confidenza oltre il quale una predizione viene considerata positiva. Nella classificazione binaria tradizionale, questa soglia è solitamente fissata a 0.5, ma può essere modificata in base al contesto. Nella *object detection*, come accennato precedentemente, oltre alla soglia di confidenza, si introduce anche una soglia sull'IoU (*Intersection over Union*) per determinare se un oggetto rilevato è da considerare un *true positive*. La variazione di queste soglie influisce direttamente sulla distribuzione dei valori all'interno della matrice di confusione e, di conseguenza, sulle metriche derivate.

5.6.1 Considerazioni sulla matrice di confusione nel caso monoclasse

Nel nostro caso, il modello è addestrato per rilevare unicamente la classe **Persona**. Non sono presenti classi negative o di background etichettate esplicitamente nei dati di ground truth, poiché nei task di object detection, tutto ciò che non è etichettato è implicitamente considerato *background*.

Di conseguenza, la **matrice di confusione** assume una struttura particolare. Essa si basa sul confronto tra le classi previste e quelle reali, ma avendo una sola classe annotata, possiamo distinguere soltanto i seguenti casi:

- **True Positive (TP)**: il modello predice una **Persona** e corrisponde a una ground truth effettivamente presente.
- **False Positive (FP)**: il modello predice una **Persona** dove non era presente alcuna persona (falso allarme).
- **False Negative (FN)**: il modello non rileva una persona presente nel ground truth.

Non esiste invece il caso di **True Negative (TN)**, poiché il modello non è progettato né addestrato per rilevare *background* come classe esplicita. In altri termini, quando un oggetto non viene predetto con confidenza sufficiente, esso è implicitamente ignorato e trattato come background, ma non viene registrato alcun confronto diretto tra background previsto e background reale, in assenza di annotazioni per il background.

Pertanto, il quadrante in basso a destra (TN) della matrice di confusione risulterà **vuoto o nullo**. Questo è un comportamento atteso nei task di rilevamento con una singola classe oggetto. Le metriche di valutazione come **precision**, **recall** e **mAP** rimangono comunque ben definite e affidabili anche in questo contesto.

Chapter 6

Risultati Sperimentali

In questo capitolo vengono presentati i risultati ottenuti durante la fase sperimentale. L'obiettivo principale è quello di valutare le prestazioni dei modelli YOLO12-n, YOLO12-s e YOLO12-m, allenati sul dataset descritto nei capitoli precedenti. Le valutazioni sono state effettuate sia durante il training (tramite le curve di loss), sia tramite metriche quantitative sul validation e test set.

6.1 YOLO12 Training Data

In questa sezione vengono presentati i risultati sperimentali ottenuti durante l'addestramento dei modelli YOLO12 nelle varianti **m**, **n** e **s**. Per ciascun modello, sono riportati i grafici delle principali *loss functions* monitorate durante le fasi di *training* e *validation*, unitamente alle rispettive **matrici di confusione** ottenute sul *validation set*.

Tali matrici consentono di visualizzare la distribuzione delle predizioni corrette ed errate effettuate dal modello per la classe **Persona**, che rappresenta l'unica classe considerata nel task di *people detection*. Le matrici sono state generate automaticamente dal framework YOLO12 utilizzando soglie di confidenza e IoU predefinite.

Influenza delle soglie decisionale.

È importante sottolineare che la costruzione delle matrici di confusione che verranno riportate di seguito dipende da una **soglia decisionale**, ovvero un

valore di confidenza oltre il quale una predizione viene considerata positiva. Nella *object detection* con YOLO12, il suo valore standard è fissato a 0.25, come da documentazione [6]. Per quanto riguarda la IoU, allo stesso modo, il valore predefinito è 0.7.

6.1.1 YOLO12-n

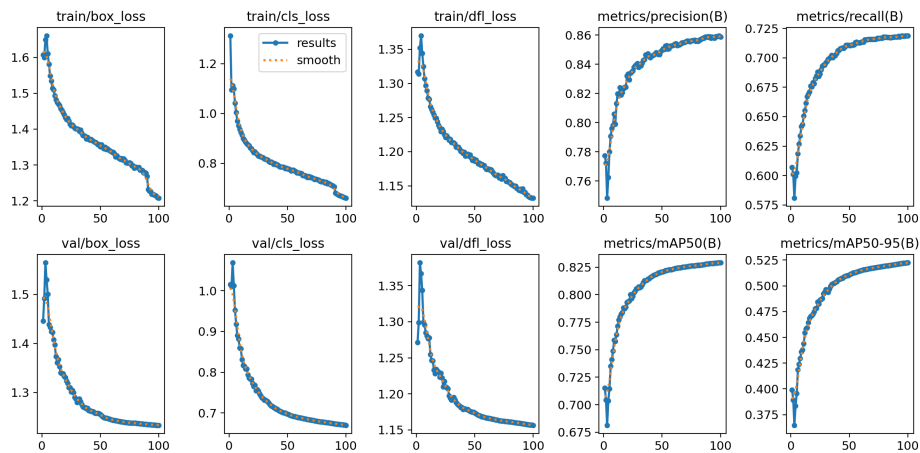


Figure 6.1: Andamento delle loss di training e validation per YOLO12-n

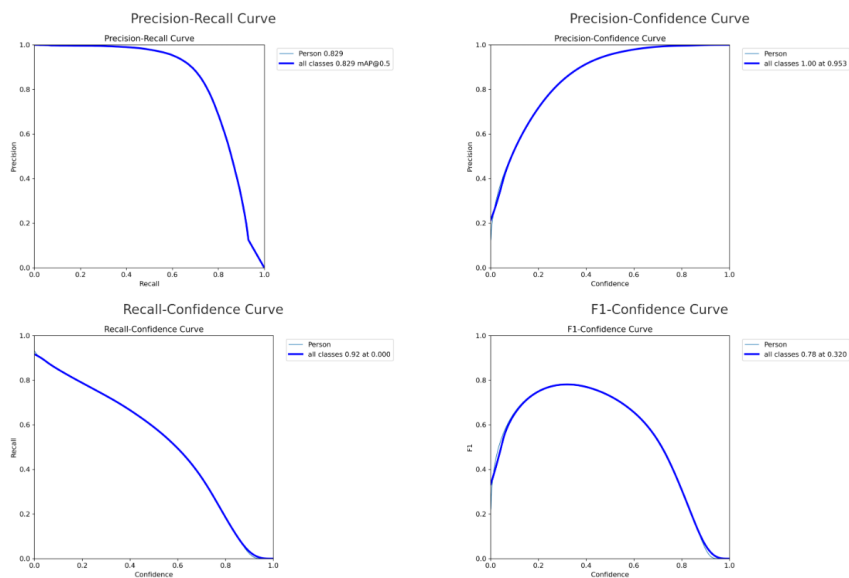


Figure 6.2: Curve di valutazione del modello YOLO12-n sulla classe Person.

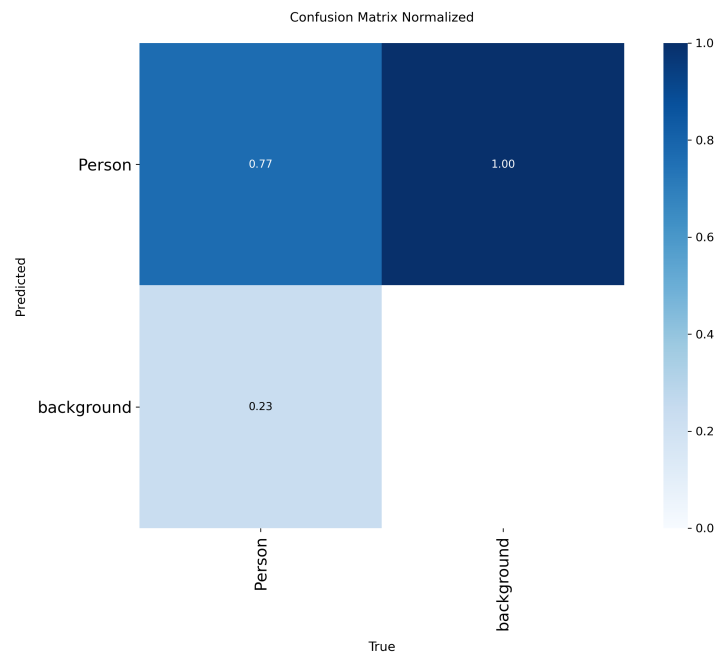


Figure 6.3: Matrice di confusione normalizzata

6.1.2 YOLO12-s

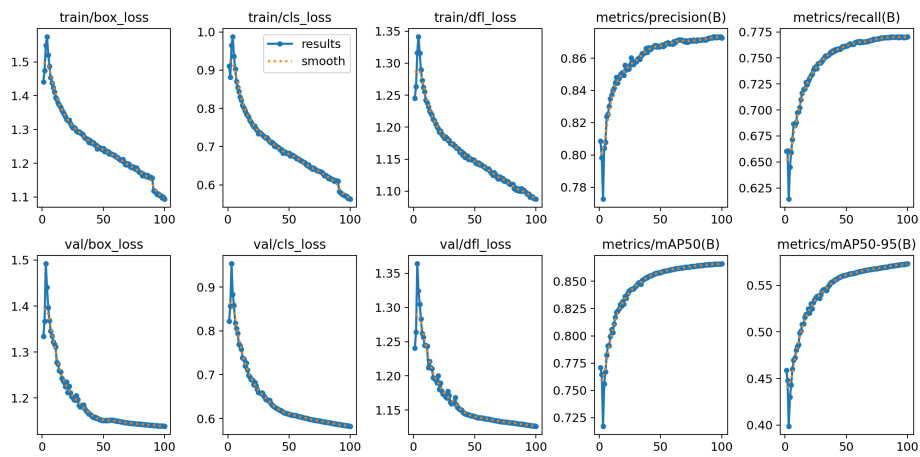


Figure 6.4: Andamento delle loss di training e validation per YOLO12-s

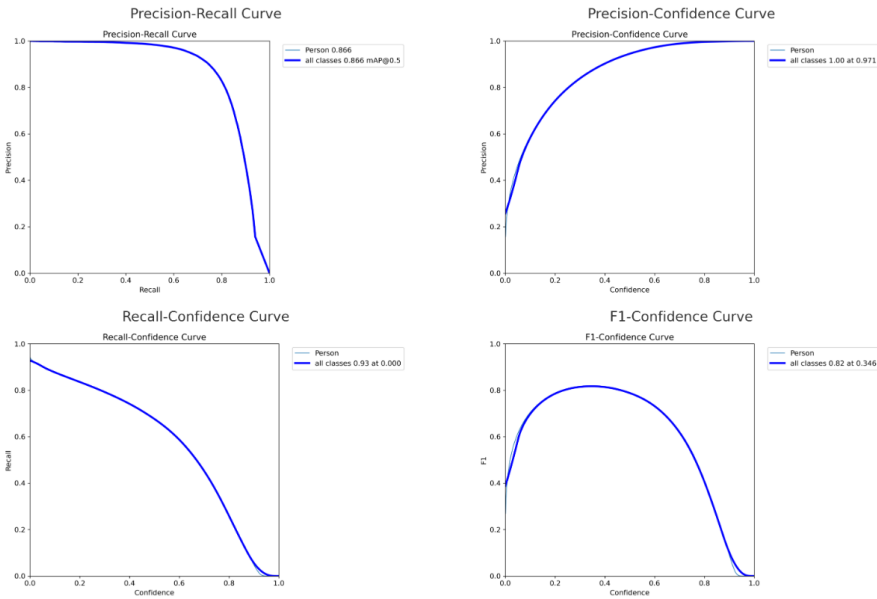


Figure 6.5: Curve di valutazione del modello YOLO12-s sulla classe Person.

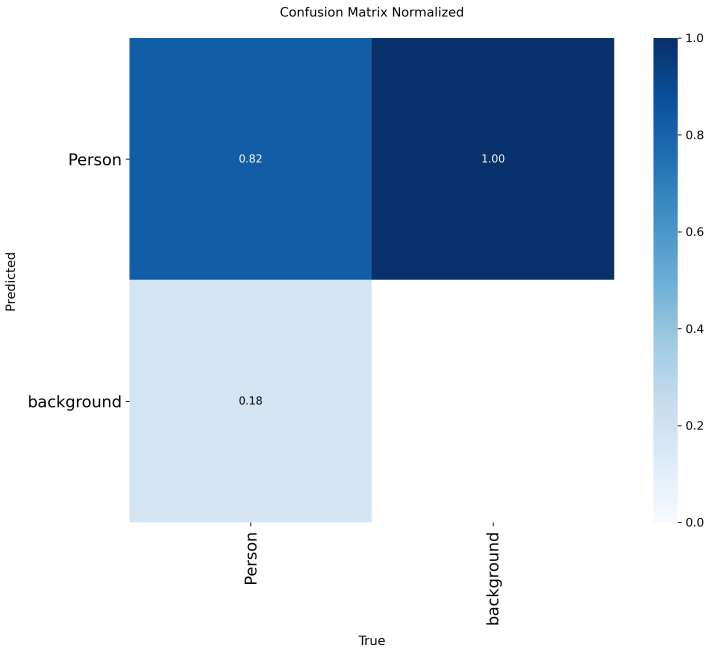


Figure 6.6: Matrice di confusione normalizzata

6.1.3 YOLO12-m

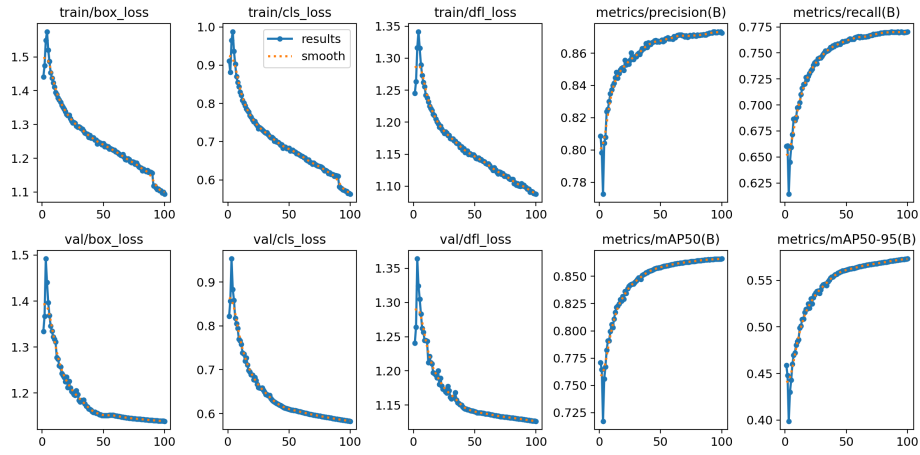


Figure 6.7: Andamento delle loss di training e validation per YOLO12-s

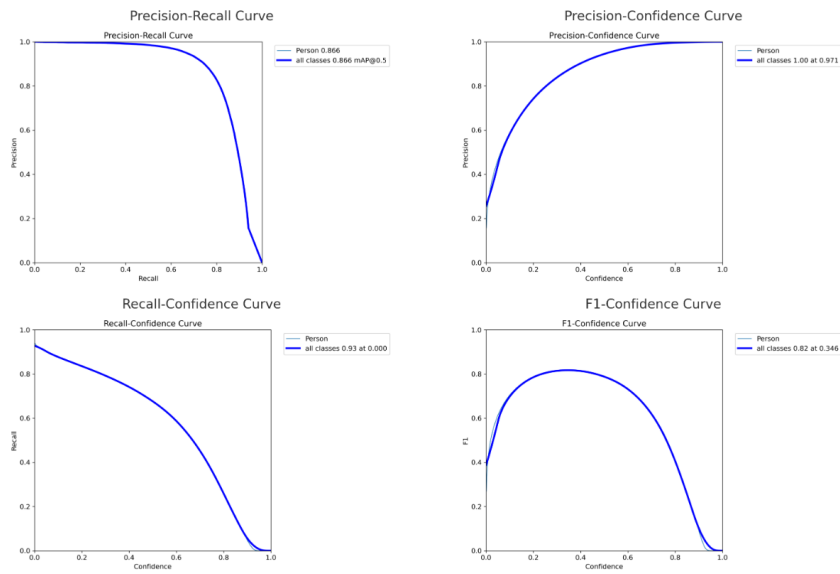


Figure 6.8: Curve di valutazione del modello YOLO12-s sulla classe Person.

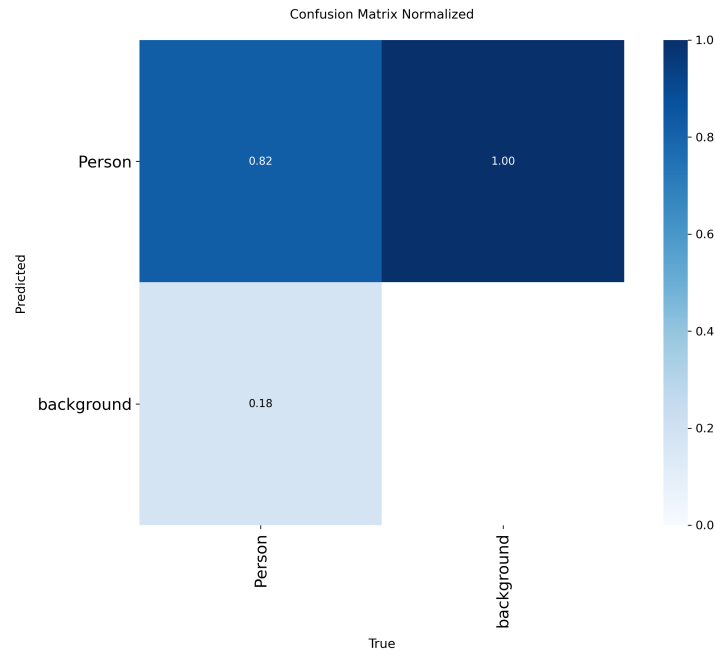


Figure 6.9: Matrice di confusione normalizzata

6.2 Confronto tra Modelli

Modello	mAP@50	mAP@50:95	Precisione	Recall
YOLO12-n	0.83	0.52	0.86	0.72
YOLO12-s	0.87	0.57	0.87	0.77
YOLO12-m	0.89	0.61	0.88	0.80

Table 6.1: Confronto delle performance sui modelli allenati

6.3 Valutazione sul Test Set

Per valutare la capacità di generalizzazione, i modelli sono stati testati su un *test set* indipendente, acquisito e annotato manualmente. Questo set include immagini in condizioni reali, non presenti nella fase di training, al fine di misurare le prestazioni in contesti nuovi e non controllati.

È stato inoltre effettuato un confronto tra i modelli YOLO12-(n,s,m) *pre-addestrati* da Ultralytics e i modelli YOLO12-(n,s,m) ottimizzati tramite fine-

tuning sui nostri dataset. L'obiettivo è valutare l'efficacia dell'adattamento specifico al task di *person counting*, confrontando approcci generici e specializzati.

Si precisa che, durante la fase di valutazione sul test set, non è stata utilizzata la modalità di inferenza basata su *SAHI* (Slicing Aided Hyper Inference). Sebbene questa tecnica sia disponibile all'interno della GUI e possa essere attivata dall'utente per ottenere prestazioni migliori nel rilevamento di oggetti di piccole dimensioni, si è scelto di escluderla dai test sperimentali per garantire uniformità tra i modelli. Inoltre, l'impiego di SAHI, pur offrendo vantaggi in scenari ad alta densità con soggetti piccoli, tende a introdurre errori in presenza di oggetti di grandi dimensioni. Di conseguenza, su immagini contenenti poche persone (ma di dimensioni rilevanti) avrebbe potuto penalizzare artificialmente le performance, compromettendo la coerenza dei confronti tra i modelli.

6.3.1 YOLO12-n e YOLO12-n (fine-tuned)

Di seguito i risultati sul test set sul modello YOLO12-n:

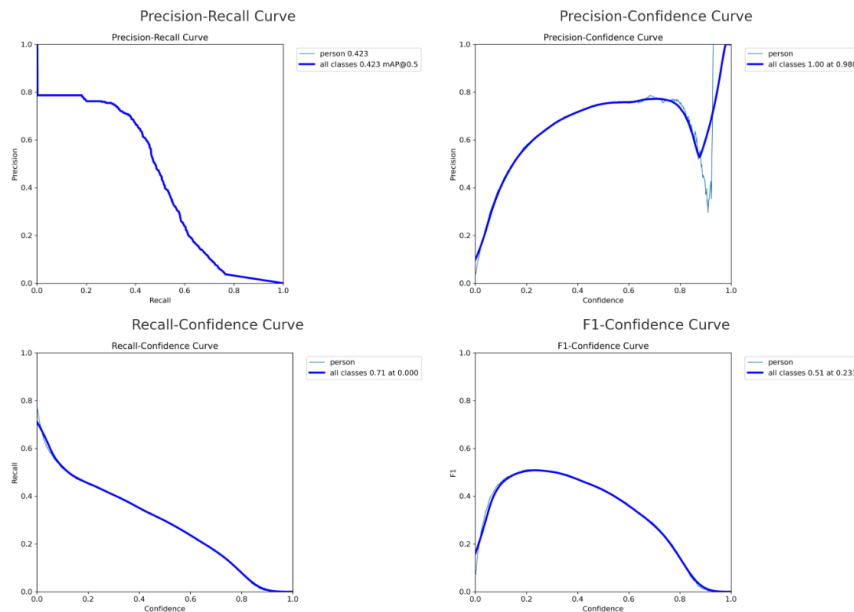


Figure 6.10: Curve di valutazione del modello YOLO12-n

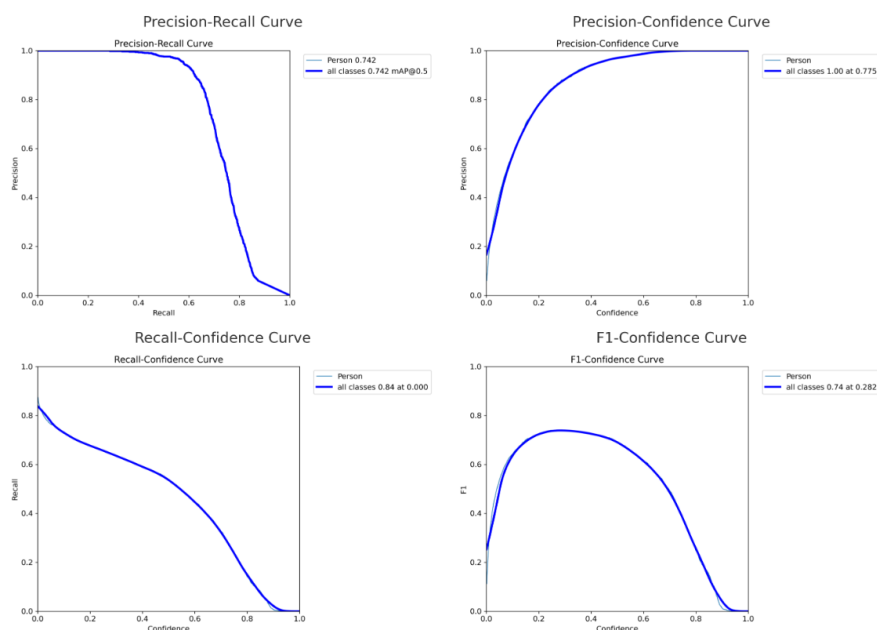


Figure 6.11: Curve di valutazione del modello YOLO12-n (fine-tuned)

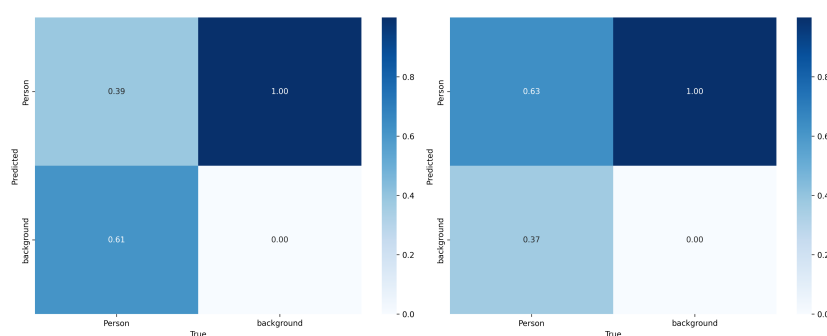


Figure 6.12: Matrici di confusione: modello YOLO12-n (a sinistra) e modello YOLO12-n (fine-tuned) a destra

Come si osserva dalle curve, il modello YOLO12-n fine-tuned mostra un miglior bilanciamento tra precision e recall rispetto al modello standard, confermando l'efficacia dell'adattamento ai dati specifici del nostro contesto.

6.3.2 YOLO12-s e YOLO12-s (fine-tuned)

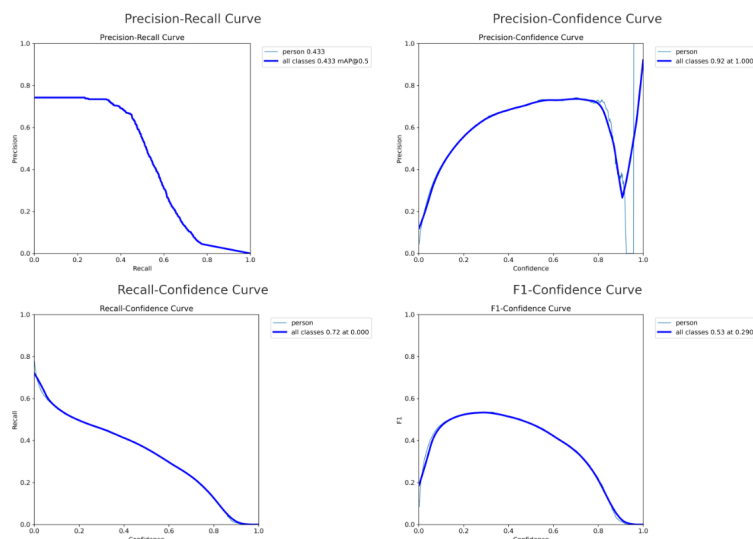


Figure 6.13: Curve di valutazione del modello YOLO12-s

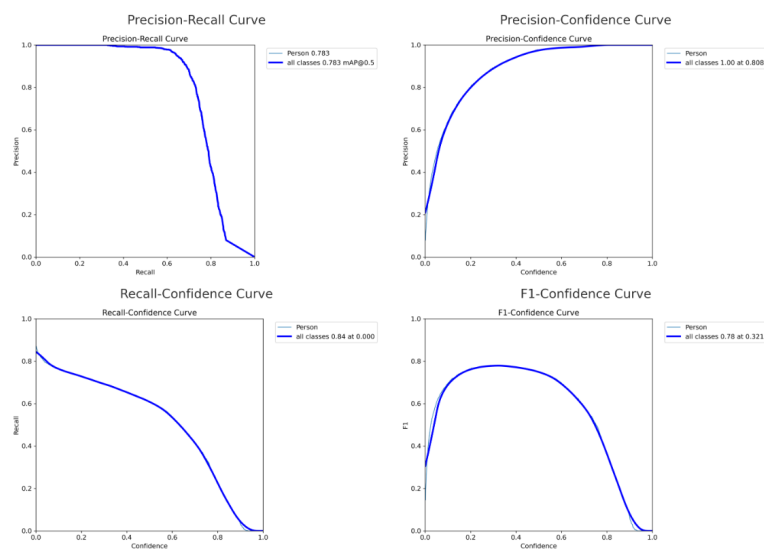


Figure 6.14: Curve di valutazione del modello YOLO12-s (fine-tuned)

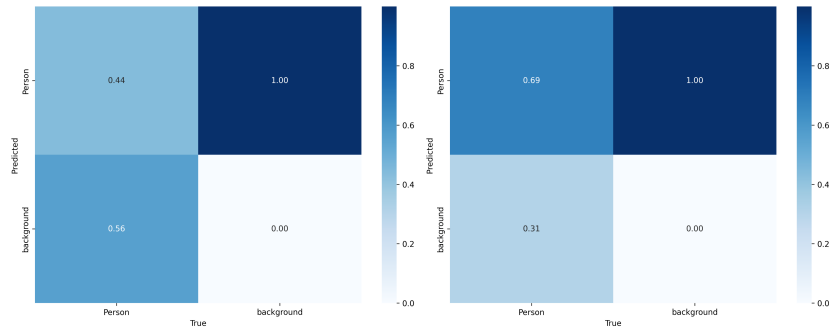


Figure 6.15: Matrici di confusione: modello YOLO12-s (a sinistra) e modello YOLO12-s (fine-tuned) a destra

Anche in questo caso con il modello fine-tuned otteniamo prestazioni nettamente migliori.

6.3.3 YOLO12-m e YOLO12-m (fine-tuned)

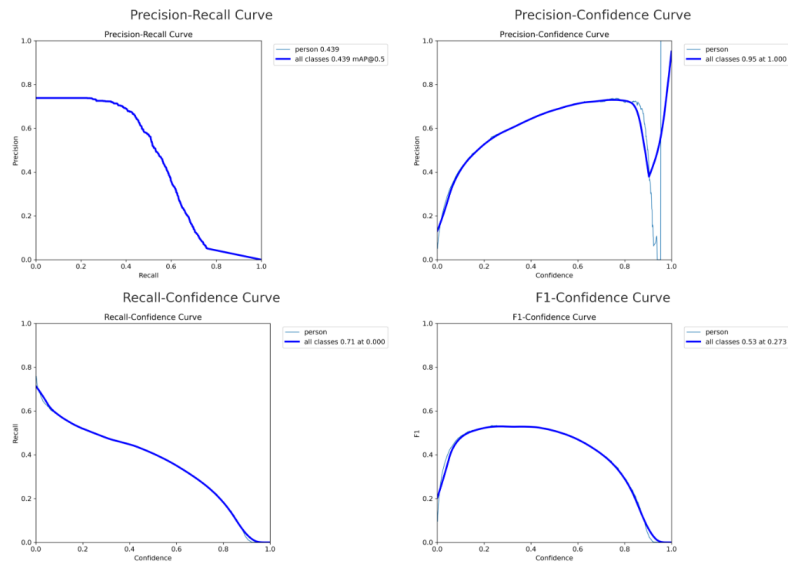


Figure 6.16: Curve di valutazione del modello YOLO12-m

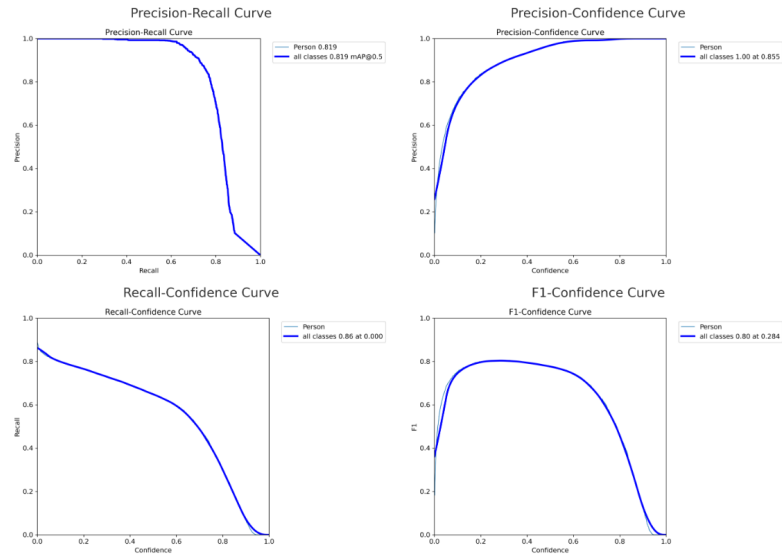


Figure 6.17: Curve di valutazione del modello YOLO12-m (fine-tuned)

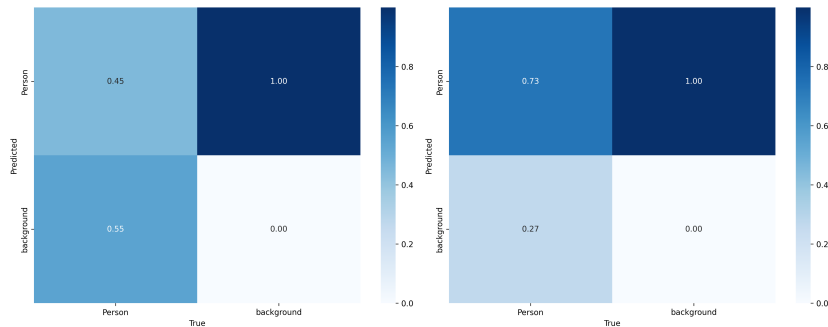


Figure 6.18: Matrici di confusione: modello YOLO12-m (a sinistra) e modello YOLO12-m (fine-tuned) a destra

Questo è il modello più grande che è stato possibile testare, ed effettivamente i risultati suggeriscono che performa meglio di tutti gli altri.

6.3.4 Tabella riepilogativa finale

Riepiloghiamo brevemente tutti i risultati ottenuti dai vari modelli e cerchiamo di trarne delle conclusioni.

Modello	Precision	Recall	F1-score	mAP@0.5	mAP@0.5:0.95
YOLO12-n	0.61	0.44	0.51	0.42	0.22
YOLO12-n (fine-tuned)	0.87	0.64	0.74	0.74	0.45
YOLO12-s	0.64	0.46	0.53	0.43	0.24
YOLO12-s (fine-tuned)	0.91	0.69	0.78	0.78	0.49
YOLO12-m	0.58	0.49	0.53	0.44	0.25
YOLO12-m (fine-tuned)	0.89	0.73	0.80	0.82	0.52

Table 6.2: Confronto tra le performance dei modelli YOLO12 pre-addestrati e fine-tuned sul test set. Le soglie di confidenza e IoU sono quelle standard di YOLO12 discusse precedentemente.

Discussione dei Risultati

Dall'analisi dei valori riportati in Tabella 6.2 si evince un netto miglioramento delle prestazioni per tutti i modelli YOLO12 dopo il processo di *fine-tuning*. In particolare:

Il modello YOLO12-n (fine-tuned) ottiene un incremento significativo sia in **precisione** (da 0.61 a 0.87) sia in **recall** (da 0.44 a 0.64), con un conseguente aumento del **F1-score** da 0.51 a 0.74.

Anche il modello YOLO12-s (fine-tuned) mostra un miglioramento marcato, passando da un F1-score di 0.53 a 0.78. Anche la precision passa da 0.64 a 0.91 e la recall da 0.46 a 0.69.

Il modello YOLO12-m (fine-tuned) è quello che raggiunge le prestazioni più elevate in assoluto, con un F1-score pari a 0.80, una precisione di 0.89 e un recall di 0.73.

Dal confronto tra le metriche **mAP@0.5** e **mAP@0.5:0.95**, si osserva come anche la capacità del modello di mantenere alte performance su più soglie di IoU sia migliorata con il fine-tuning, evidenziando un comportamento più robusto e generalizzabile. In particolare, YOLO12-m (fine-tuned) raggiunge un valore di **mAP@0.5:0.95 pari a 0.52**, superiore rispetto a tutti gli altri modelli.

Nel confronto tra i modelli si osserva inoltre che, all'aumentare della complessità architettuale (da YOLO12-n a YOLO12-m), le prestazioni migliorano sistematicamente. Questo comportamento conferma quanto ci si aspetta: modelli più grandi, pur richiedendo più risorse computazionali, ri-

escono a catturare meglio la complessità del problema, producendo predizioni più accurate e affidabili.

È interessante notare che, sebbene YOLO12-m (fine-tuned) abbia una precisione leggermente inferiore rispetto a YOLO12-s (fine-tuned) (0.89 contro 0.91), le metriche più robuste e significative come **mAP@0.5** e **mAP@0.5:0.95** evidenziano una superiorità complessiva del modello m. Questo suggerisce che, su un intervallo più ampio di soglie IoU, il modello m mantiene una performance più costante e generalizzabile, a testimonianza di una maggiore solidità nelle predizioni.

Nel complesso, si conferma che il fine-tuning è essenziale per adattare i modelli pre-addestrati al dominio specifico del task, e che metriche composite come mAP offrono una valutazione più affidabile rispetto alle sole precision, recall e F1-score, dipendenti da una soglia specifica.

Chapter 7

Demo

In questo capitolo viene presentata l'interfaccia grafica sviluppata per la dimostrazione delle funzionalità del sistema di rilevamento e conteggio delle persone. L'interfaccia è progettata per essere semplice e intuitiva, permettendo una valutazione visiva immediata delle prestazioni del modello. L'interfaccia è stata sviluppata in linguaggio Python.

7.1 Interfaccia Grafica

L'interfaccia è suddivisa in diverse aree funzionali:

- **Selezione del modello:** situata nell'angolo in alto a sinistra, consente di scegliere la variante del modello YOLO da utilizzare per l'inferenza (es. `Yolo12s`) tramite un menu a tendina.
- **Inferenza con SAHI:** una checkbox permette di attivare o disattivare l'inferenza con la libreria SAHI (Slicing Aided Hyper Inference), utile per migliorare l'accuratezza sul rilevamento di oggetti di dimensione molto piccola rispetto alla dimensione dell'immagine.
- **Selezione della dimensione per SAHI:** situata in alto a destra, consente di impostare la dimensione della finestra con cui viene suddivisa l'immagine durante l'inferenza (di default è impostata `640x640`).
- **Area di visualizzazione delle immagini/video di input, con ground truth, e di output (con predizioni).**

- **Area dei comandi principali**, che possono essere differenti sulla base del fatto che l'input sia una singola immagine oppure un video.



Figure 7.1: Interfaccia all'avvio dell'applicazione

7.1.1 Visualizzazione dei Risultati

L'interfaccia consente la visualizzazione dei dati attraverso tre pannelli principali:

- **Immagine Originale**: nel pannello di sinistra viene mostrata l'immagine o il frame originale del video, privo di annotazioni.
- **Ground Truth**: nel pannello di destra viene visualizzata la stessa immagine con i *bounding box* reali (annotazioni manuali) delle persone, usati come riferimento per il confronto. Viene inoltre mostrata una label che mostra a schermo la dimensione dei frame di un video o dell'immagine. Questo può tornare utile per cercare di capire la slicing window ottimale in base alla precisione desiderata del task.
- **Predizioni del Modello**: dopo aver effettuato l'inferenza, viene generato un terzo pannello nella sezione centrale, posizionato inferiormente rispetto ai primi due (e non visibile inizialmente), che mostra l'immagine o il video con i *bounding box* predetti dal modello.

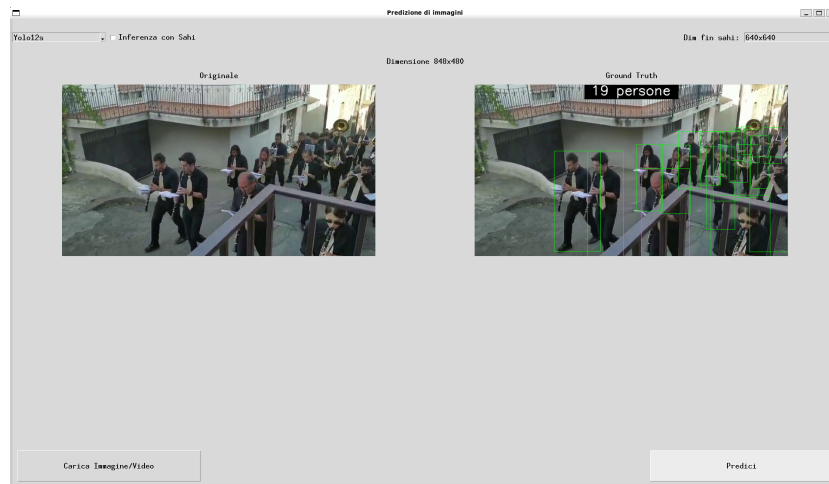


Figure 7.2: Interfaccia dopo il caricamento dell'immagine o video originale

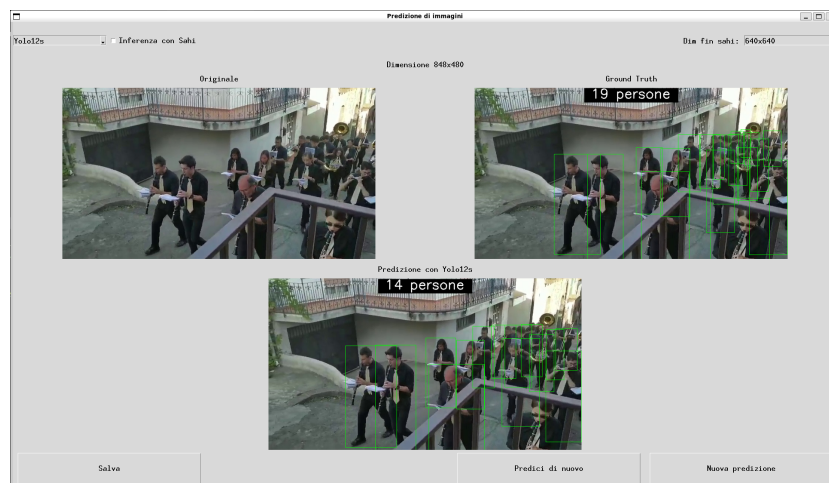


Figure 7.3: Interfaccia che mostra i risultati della predizione per le immagini

7.1.2 Controlli Principali

Nella parte inferiore dell'interfaccia sono presenti tre pulsanti funzionali:

- **Carica Immagine/Video:** consente di selezionare un file immagine o video dal dispositivo locale da utilizzare per l'inferenza.
- **Predici:** esegue l'inferenza sull'immagine o video selezionato. Una volta completata l'inferenza, il sistema mostra l'immagine o video con le predizioni del modello.

7.1.3 Controlli successivi ad una predizione

Dopo l'esecuzione di una predizione, l'interfaccia mostra tre nuovi pulsanti nella parte inferiore, che consentono di gestire e ripetere l'operazione in modo flessibile:

- **Salva:** questo pulsante consente di salvare il risultato della predizione (immagine o video) in un percorso selezionato dall'utente tramite un classico dialogo di salvataggio file. È utile per archiviare o condividere le predizioni ottenute.
- **Predici di nuovo:** consente di rieseguire l'inferenza sull'immagine o video attualmente caricato. Questo è particolarmente utile se si desidera modificare l'impostazione relativa all'uso di SAHI o alla dimensione della finestra utilizzata durante la predizione, senza dover ricaricare il file.
- **Nuova predizione:** permette di caricare una nuova immagine o un nuovo video da analizzare. Cliccando su questo pulsante si aprirà una nuova finestra di dialogo per caricare il nuovo file, successivamente si verrà riportati alla schermata iniziale con i video o immagini e le relative ground truth caricate.
- **Pause (solo per i video):** permette di mettere il video in pausa.

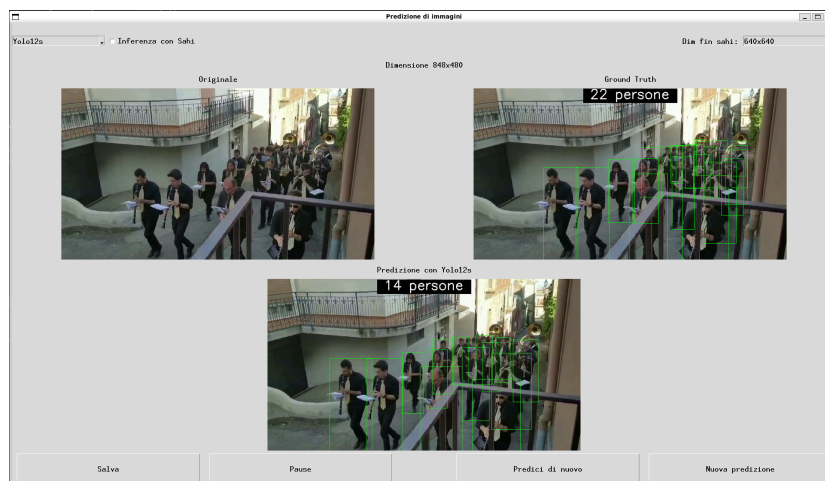


Figure 7.4: Interfaccia dei controlli successivi ad una predizione

7.1.4 Elaborazione Video

Durante l'elaborazione di un video, l'interfaccia è leggermente differente e fornisce un feedback visivo in tempo reale grazie a una barra di avanzamento (*progress bar*) che indica il numero di frame già elaborati rispetto al totale.

Nel corso della predizione, i tre pannelli principali dell'interfaccia vengono aggiornati in modo sincronizzato per ogni frame del video:

- Il primo pannello mostra il **frame originale**, senza annotazioni.
- Il secondo pannello visualizza il **frame con le ground truth**, ovvero i bounding box reali associati alle persone presenti.
- Il terzo pannello mostra il **frame con le predizioni** generate dal modello YOLO.

Questi tre pannelli vengono aggiornati simultaneamente ad ogni frame elaborato, consentendo all'utente di seguire passo passo l'andamento dell'inferenza.

Al termine dell'elaborazione, i tre video risultanti vengono riprodotti in modo sincronizzato alla velocità normale, permettendo una comparazione immediata e fluida tra il contenuto originale, le annotazioni reali e le predizioni del modello.

Controlli durante l'elaborazione dei video

Durante l'inferenza su un video, sono inoltre disponibili due pulsanti per il controllo del processo:

- **Annulla**: interrompe l'elaborazione in corso e riporta l'utente alla schermata iniziale, mantenendo però il video già caricato. Questo consente di rieseguire la predizione con configurazioni diverse senza dover ricaricare il file.
- **Ferma**: consente di interrompere anticipatamente l'inferenza. In questo caso, l'elaborazione viene fermata al frame corrente e viene generato un video contenente solo i frame elaborati fino a quel punto. Il risultato viene salvato e sincronizzato perfettamente con i corrispondenti frame nei video originali e nelle ground truth.

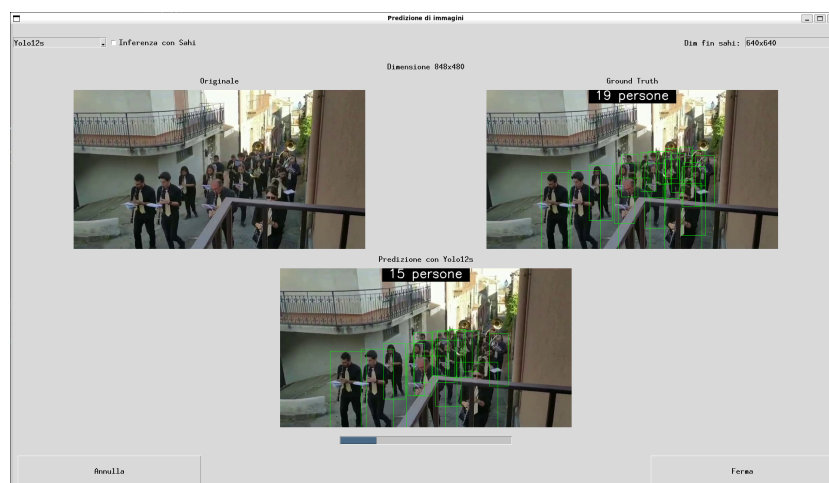


Figure 7.5: Interfaccia video durante l'elaborazione

Chapter 8

Conclusioni

In questo capitolo vengono sintetizzati i risultati principali raggiunti nel corso del progetto, le lezioni che si sono potute trarre dall'esperienza e le possibili direzioni future per lo sviluppo del sistema.

8.1 Risultati Ottenuti

L'obiettivo generale del progetto era realizzare un sistema in grado di rilevare e contare le persone presenti all'interno di video acquisiti in ambienti reali e potenzialmente complessi. A tal fine, sono stati addestrati e confrontati tre modelli della famiglia YOLO12, nelle versioni n, s e m. Ciascun modello è stato sottoposto a una fase di fine-tuning, che ha permesso di migliorarne sensibilmente le prestazioni rispetto alla versione standard, con risultati particolarmente significativi in contesti affollati o dinamici.

L'efficacia dei modelli è stata valutata attraverso diverse metriche, tra cui precision, recall e F1-score, supportate da curve di valutazione che hanno permesso di evidenziare con maggiore chiarezza il comportamento dei modelli nei vari scenari. Il modello finale si è dimostrato robusto e in grado di generalizzare bene anche su dati non presenti nei dataset di addestramento, dimostrando la validità dell'approccio adottato.

Tuttavia, è stato riscontrato che il modello presenta alcune difficoltà nel rilevamento accurato delle persone in contesti estremamente affollati, in cui molti soggetti risultano ammassati e sovrapposti. Questo limite è riconducibile a due principali fattori: da un lato, le restrizioni dell'hardware uti-

lizzato (GPU con 12GB di VRAM), che impongono vincoli sulla risoluzione elaborabile e sulla complessità computazionale; dall'altro, la natura del dataset di training, che include principalmente immagini in cui le persone sono etichettate *full body* e che privilegia contesti mediamente affollati, per favorire la generalizzazione del modello. Di conseguenza, il modello tende a perdere accuratezza proprio nei casi più critici, dove la capacità di distinguere soggetti adiacenti è fondamentale. Questo rappresenta un possibile margine di miglioramento per futuri sviluppi, sia in termini di qualità del dataset che di potenza computazionale disponibile.

Va inoltre osservato che, in scenari estremamente densi come quelli tipici del *crowd counting*, risulta più efficace adottare un diverso approccio di annotazione, in cui si etichettano solamente le teste degli individui. Infatti, in condizioni di alta densità, le forti sovrapposizioni rendono spesso non visibili intere porzioni del corpo, mentre la testa rimane la parte più distintiva e frequentemente rilevabile. Una tale strategia potrebbe migliorare significativamente la precisione del conteggio in scene particolarmente affollate.

8.2 Lezioni Apprese

Lo sviluppo del progetto ha permesso di acquisire una serie di conoscenze tecniche e metodologiche di rilievo. Innanzitutto, è emersa l'importanza cruciale del preprocessing, in particolare nella trasformazione dei dataset, nella conversione delle bounding box e nella corretta selezione delle classi da includere. Un altro fattore importante è la costruzione del dataset che incide sul fine-tuning al fine di adattare efficacemente i modelli al compito specifico del conteggio delle persone.

Infine, il progetto ha mostrato come il rilevamento di soggetti piccoli o parzialmente occlusi rimanga una sfida aperta, richiedendo particolare attenzione nella scelta e nella preparazione dei dataset.

8.3 Sviluppi Futuri

Il lavoro svolto costituisce una base solida per lo sviluppo di ulteriori miglioramenti. Un primo passo potrebbe consistere nell'integrazione di un sistema

di tracking multi-oggetto, come ad esempio Deep SORT, al fine di ottenere un conteggio temporale più stabile e accurato. Sarebbe inoltre possibile procedere con ulteriori fasi di fine-tuning mirate a contesti specifici, come ambienti industriali, stazioni ferroviarie o eventi ad alta densità di persone. Un altro miglioramento potrebbe essere utilizzare delle risoluzioni maggiori in fase di training, anche se questo richiederebbe una potenza computazionale superiore. Infine, si potrebbe estendere il sistema al riconoscimento e conteggio di più classi di oggetti, rendendolo utile in scenari applicativi ancora più ampi.

Chapter 9

Codice

In questo capitolo viene descritto come è organizzato il codice del progetto e come è possibile eseguirlo correttamente a partire dalla repository remota.

Il codice è disponibile su una repository GitHub. Per poterlo eseguire, è sufficiente clonare il progetto utilizzando i comandi forniti da GitHub. Una volta completata la clonazione, la struttura della cartella sarà la seguente:

```
├── script/                                #contains the python script
│   ├── _pycache/
│   ├── gui_utils/
│   │   ├── cache/
│   │   │   ├── weigths/
│   │   │   │   ├── our_yolo12m.pt
│   │   │   │   ├── our_yolo12n.pt
│   │   │   │   └── our_yolo12s.pt
│   │   └── CUI%_to_yolo.py
│   ├── _init_.py
│   ├── confusion_matrix_for_yolo.py
│   ├── crowd-human-to-yolo.py
│   ├── generate_heatmap.py
│   ├── mot_to_yolo.py
│   ├── predict_for_gui.py
│   ├── test.py
│   ├── training.py
│   ├── utils.py
│   └── yolo11n.pt
├── test_result                            #contains the valutation on test set
│   ├── our_yolo_m
│   ├── our_yolo_n
│   ├── our_yolo_s
│   ├── yolo_m
│   ├── yolo_n
│   └── yolo_s
├── training_result                        #contains the result on training
│   ├── Yolo12m
│   │   └── weights
│   ├── Yolo12n
│   │   └── weights
│   └── Yolo12s
│       └── weights
├── video_test                            #contains video for testing
├── demo.mp4                             #short video that explains how to use the app
├── demo.py                               #app that allows you to do inference
├── download_dataset.py                   #script that allows you to download the dataset
├── Relazione_finale_ML.doc               #final relation of the project in doc format
├── Relazione_finale_ML.pdf               #final relation of the project in pdf format
└── requirements.txt                       #contains the dependencies
```

Figure 9.1: Struttura della repository principale

Dopo essersi spostati all'interno della cartella principale della repository tramite terminale, è necessario installare tutte le dipendenze richieste eseguendo il comando:

```
pip install -r requirements.txt
```

Questo comando provvede a installare tutte le librerie necessarie all'avvio e al funzionamento del progetto.

Una volta installate le dipendenze, è necessario scaricare il dataset, il quale è ospitato su Google Drive. Per farlo, è sufficiente eseguire il comando:

```
python download_dataset.py
```

Lo script utilizza la libreria **gdown** per scaricare automaticamente il dataset e salvarlo nella cartella della repository, includendo le immagini di training, validation e test. Tuttavia, si segnala che in alcuni casi **gdown** potrebbe non riuscire a completare il download a causa di limitazioni imposte da Google Drive (es. limite di traffico superato). In tali situazioni, il percorso diretto del file verrà comunque stampato a schermo dallo script, permettendo all'utente di scaricarlo manualmente e di estrarlo manualmente nella stessa directory della repository.

Una volta completata anche questa operazione, il sistema è pronto per eseguire le predizioni. È sufficiente avviare il seguente comando da terminale:

```
python demo.py
```

Il tool caricherà automaticamente il modello e applicherà il processo di rilevamento e conteggio delle persone sulle immagini di test.

Organizzazione degli Script

All'interno della cartella **script/** si trovano tutti gli script necessari per la conversione dei dataset, l'addestramento, l'inferenza e l'analisi dei risultati.

Gli script di conversione sono collocati direttamente nella cartella **script/** e presentano, all'inizio del codice, una serie di percorsi predefiniti che puntano alle directory contenenti le immagini, le annotazioni o altri elementi

necessari al funzionamento. Tali percorsi sono chiaramente indicati e facilmente modificabili. Per un utilizzo personale, è sufficiente aggiornare i path in base alla propria struttura locale ed eseguire gli script desiderati con il comando:

```
python nome_dello_script.py
```

Il file `utils.py` contiene una serie di funzioni di supporto utilizzate dagli altri script, come la generazione automatica delle bounding box ground truth o la creazione della struttura delle directory secondo il formato richiesto da YOLO.

Il file `predict_for_gui.py` implementa i metodi di inferenza utilizzati all'interno della demo, sia nella modalità standard che in quella estesa con l'ausilio della libreria *Sahi*, utile per migliorare il rilevamento su immagini ad alta risoluzione mediante il metodo dello slicing.

I file `test.py` e `confusion_matrix_for_yolo.py` sono stati impiegati per la generazione delle statistiche di valutazione dei modelli sul dataset di test, includendo il calcolo delle metriche e la creazione delle curve di precisione, recall, F1-score e delle matrici di confusione.

Il file `generate_heatmap.py` permette di generare le heatmap dei dataset.

Infine, il file `training.py` contiene lo script principale utilizzato per il fine-tuning dei modelli YOLO12, con il caricamento dei pesi pre-addestrati e la configurazione dei parametri di addestramento specifici per il task di people counting.

Nell'ultima parte della relazione sono elencati gli script principali che hanno portato alla realizzazione del progetto, tuttavia, per brevità, non sarà esaustiva, nè conterrà tutti gli script utilizzati. Tutto il materiale mancante è comunque disponibile nella repository ufficiale del progetto.

Appendix A

Script di Conversione per il Dataset CrowdHuman

Lo script riportato di seguito è stato utilizzato per convertire il dataset CrowdHuman in formato YOLO, mantenendo solo le bounding box del corpo intero (full_body). I box sono stati normalizzati in base alla dimensione dell'immagine e corretti per rientrare interamente nel frame.

```
1 import os
2 import json
3 from PIL import Image
4 import shutil
5 from tqdm import tqdm
6 from utils import create_dataset_structure, clamp
7
8 input_dataset_dir = "../imagesCrowd/"
9 output_dataset_dir = "../dataset"
10
11 train_dir_images, train_dir_labels, val_dir_images,
12   val_dir_labels =
13   create_dataset_structure(output_dataset_dir)
14 class_id = 0
15
16 train_annotations = []
17 val_annotations = []
18
19 for file in os.listdir(input_dataset_dir):
20     if file.endswith(".odgt"):
21         full_path = os.path.join(input_dataset_dir, file)
22         with open(full_path, 'r') as f:
23             if "val" in file:
24                 val_annotations.extend(f.readlines())
25             else:
```

```

24         train_annotations.extend(f.readlines())
25
26 annotations = [train_annotations, val_annotations]
27 for ann in annotations:
28
29     desc = "Elaborazione validation"
30     label_dir = val_dir_labels
31     image_dir = val_dir_images
32
33     if ann == train_annotations:
34         desc = "Elaborazione training"
35         label_dir = train_dir_labels
36         image_dir = train_dir_images
37
38     for line in tqdm(ann, desc):
39         entry = json.loads(line.strip())
40
41         img_name = entry["ID"] + ".jpg"
42         img_path = os.path.join(input_dataset_dir, "Images",
43                                 img_name)
44
45         if not os.path.isfile(img_path):
46             continue
47
48         img = Image.open(img_path)
49         img_w, img_h = img.size
50
51         label_file = os.path.splitext(img_name)[0] + ".txt"
52         label_path = os.path.join(label_dir, label_file)
53         shutil.copy(img_path, image_dir)
54
55         with open(label_path, 'w') as out_f:
56             for box in entry["gtboxes"]:
57                 if box.get("extra", {}).get("ignore", 0) ==
58                     1:
59                     continue
60                 x, y, w, h = box["fbox"]
61                 x, y, w, h = clamp(x, y, w, h, img_w, img_h)
62                 x_center = (x + w / 2) / img_w
63                 y_center = (y + h / 2) / img_h
64                 w_norm = w / img_w
65                 h_norm = h / img_h
66                 out_f.write(f"{class_id} {x_center:.6f}
67                             {y_center:.6f} {w_norm:.6f}
68                             {h_norm:.6f}\n")

```

Listing A.1: Script di conversione da CrowdHuman a YOLO

Appendix B

Script di Conversione per il dataset CUHk

```
1 import os
2 import cv2
3 import scipy
4 import scipy.io
5 import numpy as np
6 from utils import create_dataset_structure
7
8 script_dir = os.path.dirname(os.path.abspath(file))
9
10 INPUT_CUHK_DATASET =
11     os.path.join(script_dir, '..', 'CHUK_dataset')
12 OUTPUT_YOLO_DATASET = os.path.join(script_dir, '..', 'dataset')
13
14 def get_frame_count(video_path):
15     cap = cv2.VideoCapture(video_path)
16     if not cap.isOpened():
17         return 0
18     count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
19     cap.release()
20     return count
21
22 def parse_mat_vbb(vbb_path, img_width, img_height):
23     mat = scipy.io.loadmat(vbb_path)
24     vbb = mat['A'][0, 0]
25
26     objLists = vbb['objLists']
27     n_frames = objLists.shape[1]
28     annotations = {}
29
30     for frame_idx in range(n_frames):
31         objs = objLists[0, frame_idx]
```

[illegible]


```

76         cv2.imwrite(img_save, frame)
77
78         txt_save = os.path.join(output_txt_dir,
79                                 f"{v_name}_{frame_idx:04d}.txt")
80         with open(txt_save, 'w') as f:
81             for cls, xc, yc, w, h in
82                 annotations.get(frame_idx, []):
83                 f.write(f"{cls} {xc:.6f} {yc:.6f} {w:.6f}
84                         {h:.6f}\n")
85
86         cap.release()
87         print(f"Processati {frame_idx+1} frame da {video_path}")
88
89 if __name__ == "__main__":
90
91     video_folder = os.path.join(INPUT_CUHK_DATASET, "videos")
92     vbb_folder = os.path.join(INPUT_CUHK_DATASET, "labels")
93
94     train_image_folder, train_labels_folder,
95     val_image_folder, val_labels_folder =
96     create_dataset_structure(OUTPUT_YOLO_DATASET)
97
98     video_files = [
99         f for f in os.listdir(video_folder)
100         if f.lower().endswith(".mp4")
101     ]
102     video_files.sort(key=lambda f:
103                     get_frame_count(os.path.join(video_folder, f)),
104                     reverse=True)
105     i = 0
106     for fname in video_files:
107         if not fname.lower().endswith(".mp4"):
108             continue
109         video_path = os.path.join(video_folder, fname)
110         base = os.path.splitext(fname)[0]
111         vbb_path = os.path.join(vbb_folder, base + ".vbb")
112         if not os.path.exists(vbb_path):
113             print(f"Attenzione: manca {vbb_path}")
114             continue
115         output_img_dir = val_image_folder if i==0 or i==7
116             else train_image_folder
117         output_folder_label = val_labels_folder if i==0 or
118             i==7 else train_labels_folder
119         extract_frames_and_bbox(
120             video_path,
121             vbb_path,
122             output_img_dir,
123             output_folder_label,
124             base

```

```
116 )  
117 i += 1
```

Listing B.1: Script di conversione da CUHk a YOLO

Appendix C

Script di Conversione per il dataset MOT

```
1 import os
2 import shutil
3 import pandas as pd
4 from enum import Enum
5 from collections import defaultdict
6 from PIL import Image
7 from utils import clamp, create_dataset_structure
8 import cv2
9 from tqdm import tqdm
10
11 script_dir = os.path.dirname(os.path.abspath(__file__))
12
13 MOT_DATASET_ROOT = os.path.join(script_dir, '..', 'MOT')
14 OUTPUT_YOLO_DATASET = os.path.join(script_dir, '..', 'dataset')
15
16 class Object_GT_Code(Enum):
17
18     Pedestrian = 1
19     Occupant = 2
20     StationaryPerson = 7
21
22 def convert_mot_to_yolo_single_folder(dataset_root,
23                                     output_root, objectToFilter, visibility_threshold=0.3,
24                                     frame_to_skip = 12 ):
25
26     images_output, labels_output, val_images_output,
27     val_labels_output = create_dataset_structure(output_root)
28
29     sequences = [seq for seq in os.listdir(dataset_root) if
30                  (os.path.isdir(os.path.join(dataset_root, seq))) ]
31     video_for_val = ["MOT17-04-FRCNN", "MOT17-02-FRCNN"]
```

```

29 original_w=1920
30 original_h=1080
31 for seq in sequences:
32
33     seq_path = os.path.join(dataset_root, seq)
34     img_path = os.path.join(seq_path, 'img1')
35     gt_path = os.path.join(seq_path, 'gt', 'gt.txt')
36     val_path = os.path.join(seq_path)
37
38     if not os.path.isfile(gt_path) or not
39         os.path.isdir(img_path):
40         continue
41
42     df = pd.read_csv(gt_path, header=None)
43     df.columns = ['frame', 'id', 'x', 'y', 'w', 'h',
44         'conf', 'class', 'vis']
45     df = df[(df['class'].isin([object.value for object
46         in objectToFilter])) & (df['vis'] >
47         visibility_threshold)]
48     df = df[df['frame'] % frame_to_skip ==
49         0].reset_index(drop=True)
50
51     for _, row in tqdm(df.iterrows(), total=len(df),
52         desc=f"Processing {seq}"):
53         frame_id = int(row['frame'])
54         image_name = f"{frame_id:06d}.jpg"
55         image_src_path = os.path.join(img_path,
56             image_name)
57
58         new_image_name = f"{seq}_{frame_id:06d}.jpg"
59         new_label_name = new_image_name.replace('.jpg',
60             '.txt')
61
62         new_image_path = os.path.join(images_output,
63             new_image_name) if seq not in video_for_val
64         else os.path.join(val_images_output,
65             new_image_name)
66
67         if not os.path.exists(new_image_path):
68             img = cv2.imread(image_src_path)
69             if img is None:
70                 print(f"Failed to load image:
71                     {image_src_path}")
72                 continue
73             original_h, original_w = img.shape[:2]
74
75             cv2.imwrite(new_image_path, img)

```

```

65     x_new, y_new, middlew, middleh =
        clamp(row['x'], row['y'], row['w'], row['h'],
66             img_width=original_w, img_height=original_h)
67     if middlew <= 0 or middleh <= 0:
68         print(f"Skipped box with non-positive size:
            {middlew}, {middleh}")
69         continue
70
71     x_center = (x_new + middlew / 2) / original_w
72     y_center = (y_new + middleh / 2) / original_h
73     if (x_new + middlew > original_w):
74         middlew = original_w - x_new
75     if (y_new + middleh > original_h):
76         middleh = original_h - y_new
77
78     w = middlew / original_w
79     h = middleh / original_h
80     class_label = 0
81
82     label_path = os.path.join(labels_output,
        new_label_name) if seq not in video_for_val
        else os.path.join(val_labels_output,
        new_label_name)
83
84     with open(label_path, 'a') as f:
85         f.write(f"{class_label} {x_center:.6f}
            {y_center:.6f} {w:.6f} {h:.6f}\n")
86
87     print("Conversion completed!")
88
89 convert_mot_to_yolo_single_folder(
90     dataset_root=MOT_DATASET_ROOT,
91     output_root=OUTPUT_YOLO_DATASET,
92     visibility_threshold=0.0,
93     objectToFilter = [Object_GT_Code.Pedestrian,
        Object_GT_Code.Occupant,
        Object_GT_Code.StationaryPerson]
94 )

```

Listing C.1: Script di conversione da MOT a YOLO

Appendix D

Generazione delle heatmap per training e validation

```
1 import os
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from glob import glob
6 from tqdm import tqdm
7
8
9 dataset_path = "dataset"
10 subfolders = ["train", "val"]
11 canvas_size = (1280, 1280)
12 heatmap = np.zeros(canvas_size, dtype=np.float32)
13 image_exts = [".jpg", ".jpeg", ".png"]
14
15
16 def add_bbox_to_heatmap(xc, yc, w, h, img_w, img_h):
17     x = int(xc * canvas_size[1])
18     y = int(yc * canvas_size[0])
19     sigma = int(min(w * img_w, h * img_h)) or 2
20     sigma = int(sigma * (canvas_size[0] / img_h))
21     sigma = max(sigma, 2)
22     patch = np.zeros(canvas_size, dtype=np.float32)
23     cv2.circle(patch, (x, y), sigma, 1, -1)
24     heatmap[:] += patch
25
26 for subset in subfolders:
27     label_dir = os.path.join(dataset_path, "labels", subset)
28     image_dir = os.path.join(dataset_path, "images", subset)
29     label_files = glob(os.path.join(label_dir, "*.txt"))
30
```

```

31     for label_path in tqdm(label_files, desc=f"Processing
    {subset}"):
32         filename =
            os.path.basename(label_path).replace(".txt", "")
33
34         image_path = None
35         for ext in image_exts:
36             candidate = os.path.join(image_dir, filename +
                ext)
37             if os.path.exists(candidate):
38                 image_path = candidate
39                 break
40         if not image_path:
41             continue
42
43         image = cv2.imread(image_path)
44         if image is None:
45             continue
46         img_h, img_w = image.shape[:2]
47
48         with open(label_path, "r") as f:
49             for line in f:
50                 parts = line.strip().split()
51                 if len(parts) != 5:
52                     continue
53                 _, xc, yc, w, h = map(float, parts)
54                 add_bbox_to_heatmap(xc, yc, w, h, img_w,
                    img_h)
55
56         heatmap = cv2.GaussianBlur(heatmap, (0, 0), sigmaX=3,
            sigmaY=3)
57         heatmap_norm = heatmap / heatmap.max()
58         plt.figure(figsize=(8, 8))
59         plt.imshow(heatmap_norm, cmap="hot")
60         plt.axis("off")
61         plt.title("Heatmap aggregata delle annotazioni
            (normalizzata)")
62         plt.tight_layout()
63         plt.savefig(f"heatmap_annotazioni_{subset}.png", dpi=300)
64         heatmap = np.zeros(canvas_size, dtype=np.float32)

```

Listing D.1: Script per la generazione delle heatmap per training e validation

Appendix E

Procedura di Training

```
1
2 import sys
3 import os
4
5 from ultralytics import YOLO
6
7 def main():
8     model = YOLO('yolo12n.pt')
9     results = model.train(
10         data='../dataset/dataset.yaml',
11         epochs=100,
12         batch=12,
13         imgsz=640,
14         scale=0.9,
15         mosaic=0.4,
16         close_mosaic = 10,
17         device="0",
18         workers = 4,
19         amp = True,
20         patience=20,
21         verbose=True,
22         cache = True,
23         name="Dataset-YOLO12s-640-crowd+Mot",
24         resume = False
25     )
26
27
28 if name == 'main':
29     from multiprocessing import freeze_support
30     freeze_support()
31     main()
```

Listing E.1: Procedura di Training

Bibliography

- [1] MOTChallenge, “Motchallenge benchmark,” 2025. [Online]. Available: <https://motchallenge.net>
- [2] CrowdHuman Dataset, “Crowdhuman: A benchmark for detecting humans in a crowd,” 2025. [Online]. Available: <https://www.crowdhuman.org>
- [3] X. Wang *et al.*, “Cuhk pedestrian dataset,” https://www.ee.cuhk.edu.hk/~xgwang/CUHK_pedestrian.html, 2009.
- [4] R. Universe, “Crowd typsa dataset,” <https://universe.roboflow.com/razor-zdbsr/crowd-typsa>, 2023.
- [5] F. C. Akyon, S. O. Altinuc, and A. Temizel, “Slicing aided hyper inference and fine-tuning for small object detection,” *arXiv preprint arXiv:2202.06934*, 2022.
- [6] Ultralytics, “Yolov12 models — ultralytics documentation,” 2025. [Online]. Available: <https://docs.ultralytics.com/it/models/yolo12/>